

MethylPipeline: Theory, Workflows, and Reference

Mathematical foundations, operational workflows, and complete configuration reference

MethylPipeline Team

2026-04-01

Table of contents

1	Scope, Traceability, and Notation	7
1.1	Purpose	7
1.2	What Counts As “Theory” In This Repository	7
1.3	Package Map	8
1.4	Shared Data Model	9
1.5	Shared Notation	9
1.6	Cross-Cutting Assumptions	10
1.7	Reading This Book	10
1.7.1	Package Map (updated)	11
I	Part I: Mathematical and Statistical Foundations	13
2	MethylUtils	14
2.1	Scope	14
2.2	Implemented Method Inventory	14
2.3	Centroid Sufficient Statistics	15
2.4	ECDF Reconstruction	15
2.5	Two-Sample Comparison	16
2.5.1	KS Statistic on Reconstructed ECDFs	16
2.5.2	Mann-Whitney from Histogram Counts	16
2.5.3	Heterogeneity	17
2.6	Continuous Overlap	17
2.7	Multiple Testing and p-Value Aggregation	17
2.8	Canonical Biological Effect Size	18
2.9	ECDFClassifier	18
2.10	Auxiliary Beta-Based Metrics	19
3	MethylCentroid	20
3.1	Role	20
3.2	What A Centroid Stores	20
3.3	Sample-To-Centroid Update Rule	20
3.4	Coverage Capping By Binomial Thinning	21
3.5	Auto-Cap Heuristic	21
3.6	Why Histograms Matter	22
3.7	Publication Guidance	22
4	MethylDetector	23
4.1	Role	23
4.2	Alignment And Candidate Set	23

4.3	Statistical Screening	23
4.4	Biological Ranking	24
4.5	Cumulative Effect Coverage	24
4.6	Context And Chromosome Granularity	24
4.7	Exported Classifier Artifacts	25
4.8	Discovery versus prediction exports	25
4.9	Production Freeze Panel	25
4.10	DMP Coverage in New Samples	26
4.11	Publication Guidance	27
5	MethylClassifier	28
5.1	Role	28
5.2	Binary Classification	28
5.3	Multi-Chromosome Binary Classification	28
5.4	One-vs-Rest (OvR) Multiclass Fusion	29
5.5	Pairwise Control Aggregation	29
5.6	Bipartite Multi-Control $\times$ Multi-Disease Aggregation	29
5.7	Calibration	30
5.8	New Modeling Features in Production	30
5.8.1	Learned Multiclass Head	30
5.8.2	Isotonic and Stacking Layers	31
5.8.3	Practical Reproducibility	31
5.9	Multiclass Learned Head: Class Weighting	31
5.10	Missing Data Semantics	32
5.11	Publication Guidance	32
6	MethylPredictor, MethylValidation, and MethylDiseaseProgression	33
6.1	Role	33
6.2	MethylPredictor: Labeled Evaluation	33
6.3	Blind Prediction Summaries	34
6.4	MethylValidation: Split Strategy	34
6.5	Monte Carlo Evaluation	34
6.6	What The Validation Layer Does Not Do	35
6.7	Stability Analysis	35
6.8	Production Freeze	36
6.9	Hybrid Generative Backend (<code>model_backend=generative_hybrid</code>)	36
6.9.1	Data Contract and Bundle Reuse	36
6.9.2	Modeling Structure	37
6.9.3	Orchestration in <code>--model</code>	37
6.9.4	Configuration Surface	37
6.10	MethylDiseaseProgression Synthesis	38
6.10.1	Inputs and Stage Resolution	38
6.10.2	Entity Scores	38
6.10.3	Progression Labels	38
6.10.4	Outputs	39
6.11	Two Workflows Summary	39
6.12	Publication Guidance	39

7	MethylMapper	41
7.1	Role	41
7.2	Interval Mapping	41
7.3	Gene-Level Combined p-Values	41
7.4	Biological Importance Score	42
7.5	Stable Gene Set Optimization	42
7.6	External Evidence Enrichment	42
7.7	Publication Guidance	43
8	MethylEnricher	44
8.1	Role	44
8.2	Over-Representation Analysis	44
8.3	Pathway Graph Construction	44
8.4	Module Enrichment Score	45
8.5	Disease Relevance Prior	45
8.6	Theme Normalization And Labels	46
8.7	Publication Guidance	46
9	MethylAlignmentQC	47
9.1	Role	47
9.2	Deterministic Metric Extraction	47
9.3	Summary Statistics	47
9.4	Log Monitoring	47
9.5	Publication Guidance	48
10	Limitations and Open Questions	49
10.1	Why This Chapter Exists	49
10.2	The “ECDF-Only Pipeline” Claim Is Too Broad	49
10.3	Dependence Is Real	49
10.4	Approximation Layers	49
10.5	Heuristic Layers	50
10.6	External Dependencies	50
10.7	Legacy Or Experimental Remnants	50
10.7.1	Current safeguards	51
10.7.2	Deprecation Status Snapshot	51
10.8	Publication-Ready Positioning	51
10.9	What Would Strengthen The Theory Further	52
II	Part II: Workflows and Production	53
11	The Project Configuration File	54
11.1	Role and Design Principle	54
11.2	Top-Level Fields	54
11.3	The Cohort Hierarchy	55
11.4	The <code>step_config</code> Dictionary	56
11.5	The <code>step_config.predictor</code> Anti-Pattern	56

11.6	Embedding Monte Carlo Settings: <code>step_config.validation</code>	57
11.6.1	Modeling backend inside <code>step_config.validation</code>	57
11.7	Annotated Example	58
11.8	Output Directory Layout	59
12	The Two Workflows	61
12.1	Overview	61
12.2	Workflow 1: Model Creation	61
12.2.1	Theoretical Motivation	61
12.2.2	The Three Stages of Workflow 1	62
12.2.3	Steps	63
12.2.4	Workflow 1 Diagram	65
12.2.5	Expected Outputs	65
12.3	Workflow 2: Model Use for Prediction	66
12.3.1	Theoretical Motivation	66
12.3.2	Steps	66
12.3.3	Workflow 2 Diagram	67
12.4	Diagnostic: DMP Coverage	67
12.5	Diagnostic: Class Imbalance	68
12.6	Diagnostic: Sklearn Version Mismatch	68
12.7	Choosing Workflow Parameters	69
12.7.1	<code>train_fraction</code>	69
12.7.2	<code>n_iterations</code>	69
12.7.3	<code>stability_dmp_freq</code>	69
12.7.4	<code>stability_min_balanced_accuracy</code>	69
III	Part III: Configuration Reference	70
13	Configuration Reference	71
13.1	<code>step_config.centroid</code>	71
13.1.1	Batch wrapper fields	71
13.1.2	<code>base_config</code> fields (<code>MethylCentroidConfig</code>)	72
13.2	<code>step_config.detection</code> — Runtime Parameters	74
13.2.1	Input / Output	74
13.2.2	Statistical Screening	74
13.2.3	Biological Prioritization	76
13.2.4	DMP Selection and Export	78
13.2.5	Context Weighting	80
13.2.6	Classifier Handoff and Sample Coverage	81
13.2.7	Debug and Output	82
13.3	<code>step_config.detection</code> — Multiclass Export-Only Parameters	83
13.4	<code>step_config.mapper</code>	84
13.5	<code>step_config.enricher</code>	86
13.6	<code>step_config.classifier</code>	88
13.7	<code>step_config.predictor</code>	90
13.8	<code>step_config.validation</code>	91
13.9	<code>step_config.progression</code>	95

13.10	<code>step_config.alignment_qc</code>	96
13.11	Configuration Audit Matrix	96
IV	Part IV: User Guide	97
14	User Guide	98
14.1	Prerequisites	98
14.1.1	Software requirements	98
14.1.2	Environment setup	98
14.1.3	Activating the environment in subsequent sessions	99
14.2	Creating a Project Configuration File	99
14.2.1	Cohort CSV format	100
14.3	Workflow 1: Model Creation	101
14.3.1	Stage-oriented execution checklist	101
14.3.2	Step 1: Run Monte Carlo with stability	101
14.3.3	Step 2: Check stability results	102
14.3.4	Step 3: Run the production freeze	102
14.3.5	Step 4: Run the final model stage	102
14.3.6	Choosing the <code>--model</code> backend	102
14.3.7	Backend comparison (practical)	103
14.3.8	Optional: Run progression synthesis as a standalone step	104
14.4	Workflow 2: Model Use for Prediction	104
14.4.1	Prerequisites	104
14.4.2	Run predictor-only evaluation	104
14.5	Running Individual Pipeline Steps	105
14.6	Output Directory Layout	105
14.7	Interpreting Results	106
14.7.1	<code>all_metrics.csv</code>	106
14.7.2	<code>metrics_summary.json</code>	107
14.7.3	<code>validation_metrics.json</code> (per run)	107
14.7.4	Confusion matrix interpretation	108
14.8	Three Key Diagnostics	108
14.8.1	Diagnostic 1: Low DMP Coverage	108
14.8.2	Diagnostic 2: Class Imbalance	108
14.8.3	Diagnostic 3: Sklearn Version Mismatch	109
14.9	Running with Docker	109
14.10	Troubleshooting	110

1 Scope, Traceability, and Notation

1.1 Purpose

This book documents the mathematical and statistical behavior that MethylPipeline actually implements. It is not a historical reconstruction of the first design notes, and it is not a promise about future refactors. The source of truth is the current code under `packages/`.

That rule has two consequences:

1. If the code and an older markdown document disagree, the code is canonical.
2. If the code uses an approximation or heuristic, the book says so explicitly instead of rewriting it into a cleaner but false theory story.

1.2 What Counts As “Theory” In This Repository

The repository mixes several kinds of methodology. For publication-quality documentation it is important to separate them.

Category	Meaning in this book	Typical examples in MethylPipeline
Principled	Standard statistical or numerical methods used in a recognizable form	asymptotic Kolmogorov-Smirnov testing, Storey q-values, weighted log-likelihood classification, classical clustering algorithms
Approximate	A standard method implemented through discretization, asymptotics, or reconstruction from summaries rather than raw observations	histogram-based Mann-Whitney, grid-based ECDF overlap, effective-sample-size KS
Heuristic	Engineered ranking, thresholding, or aggregation rules chosen for workflow utility rather than statistical optimality	p-values effect-size ranking, module scoring constants, pairwise control aggregation

Category	Meaning in this book	Typical examples in MethylPipeline
External-service-backed	Results depend on external databases, APIs, or stored procedures whose internal logic is not fully versioned inside this repository	Enrichr, DisGeNET, Open Targets, Azure SQL stored procedures, LLM-assisted evidence synthesis

That taxonomy is used throughout the book so the reader can tell where the pipeline is closest to a classical statistical method and where it is best interpreted as a reproducible engineering decision.

1.3 Package Map

The complete package map with all four parts is in the “Reading This Book” section below. For quick orientation, Part I covers the mathematical and statistical algorithms in the following packages:

Chapter	Primary packages	Role
Chapter 2	<code>methyutils</code>	shared mathematical layer: centroids, ECDF views, testing, effect sizes, classifier densities
Chapter 3	<code>methylcentroid</code>	centroid construction and coverage preprocessing
Chapter 4	<code>methyldetector</code>	DMP testing, filtering, classifier export, and production panel bypass
Chapter 5	<code>methylclassifier</code>	binary, multi-chromosome, OvR, and multiclass learned-head classification
Chapter 6	<code>methylpredictor</code> , <code>methylvalidation</code> , <code>methyliseaseprogression</code>	prediction metrics, Monte Carlo validation, stability analysis, production freeze, progression synthesis
Chapter 7	<code>methylmapper</code>	DMP-to-gene mapping and gene-level aggregation
Chapter 8	<code>methylenricher</code>	over-representation analysis, pathway graph clustering, and module scoring
Chapter 9	<code>methylalignmentqc</code>	deterministic QC parsing and summarization
Chapter 10	<code>all</code>	caveats, legacy remnants, and claims that should be softened

1.4 Shared Data Model

At the lowest level the code works with per-position methylated and unmethylated counts. For a sample s at genomic position i ,

$$x_{si} = \frac{m_{si}}{m_{si} + u_{si}},$$

where m_{si} is the methylated count and u_{si} is the unmethylated count. The code clips or guards zero-coverage positions instead of leaving them undefined.

For a centroid with N_i contributing samples at locus i , the shared sufficient statistics are

$$N_i = \sum_s 1, \quad S_{x,i} = \sum_s x_{si}, \quad S_{x^2,i} = \sum_s x_{si}^2.$$

These yield the implemented centroid mean and unbiased sample variance:

$$\hat{\mu}_i = \frac{S_{x,i}}{N_i}, \quad \hat{\sigma}_i^2 = \max\left(\frac{S_{x^2,i} - S_{x,i}^2/N_i}{\max(N_i - 1, 1)}, \varepsilon\right),$$

with a small numerical floor ε used throughout the code to avoid degenerate densities or divisions by zero.

In the ECDF-centered path, each locus also stores a binned histogram

$$\{c_{ib}\}_{b=1}^B$$

over shared methylation bin edges $0 = e_0 < e_1 < \dots < e_B = 1$. Those histograms drive both empirical distribution reconstruction and approximate rank-based testing.

1.5 Shared Notation

Symbol	Meaning
x_{si}	methylation fraction for sample s at locus i
m_{si}, u_{si}	methylated and unmethylated counts
$N_i, S_{x,i}, S_{x^2,i}$	centroid sufficient statistics
c_{ib}	count in histogram bin b for locus i
$F_i(x)$	reconstructed cumulative distribution function at locus i
$f_i(x)$	reconstructed density, usually the derivative of a monotone spline
D_i	two-sample KS statistic at locus i
U_i	Mann-Whitney U statistic at locus i
q_i	Storey-adjusted q-value

Symbol	Meaning
O_i	continuous overlap $\int_0^1 \min(f_{1i}, f_{2i}) dx$
w_i	feature weight used by the classifier, usually derived from detector output

1.6 Cross-Cutting Assumptions

Several assumptions recur across the repository:

- Loci are treated as conditionally independent inside the classifier score, even though methylation along the genome is correlated.
- Multiple-testing corrections are applied across many dependent loci and, downstream, across aggregated genes. This is standard practice but should not be oversold as exact finite-sample control.
- Some packages operate on summaries rather than raw per-read data. That preserves scalability, but it changes the interpretation of rank tests and distribution overlap.
- Downstream packages use external services and curated priors. Those are part of the workflow, but they are not internal statistical estimators.

1.7 Reading This Book

This book is organized into four parts.

Part I: Mathematical and Statistical Foundations covers the algorithms and estimators in order from the lowest shared layer to the downstream applications. Readers interested in the production classifier should start with Chapter 2, then continue through Chapter 3, Chapter 4, and Chapter 5. Readers interested in biological interpretation should continue to Chapter 7 and Chapter 8. Chapter 10 is intentionally blunt and should be read before any publication claim is finalized.

Part II: Workflows and Production describes the two end-to-end operational workflows supported by MethylPipeline: Model Creation (repeated validation, stability analysis, production freeze, optional progression synthesis, and final `--model` step) and Model Use for Prediction (predictor-only evaluation of a frozen model). It also documents the project configuration file as the single source of truth for the entire pipeline.

Part III: Configuration Reference provides exhaustive tables of every parameter in every `step_config` section, with types, defaults, constraints, and descriptions. It is a reference, not a tutorial.

Part IV: User Guide provides step-by-step commands, output directory layout, result interpretation, and a diagnostics section covering the three most common failure modes observed in practice: low DMP coverage, class imbalance, and model version mismatch.

1.7.1 Package Map (updated)

Part	Chapter	Primary packages	Role
I	Chapter 2	<code>methylutils</code>	shared mathematical layer: centroids, ECDF views, testing, effect sizes, classifier densities
I	Chapter 3	<code>methylcentroid</code>	centroid construction and coverage preprocessing
I	Chapter 4	<code>methyldetector</code>	DMP testing, filtering, classifier export, and production panel bypass
I	Chapter 5	<code>methylclassifier</code>	binary, multi-chromosome, OvR, and multiclass learned-head classification
I	Chapter 6	<code>methylpredictor</code> , <code>methylvalidation</code> , <code>methyliseaseprogression</code>	prediction metrics, Monte Carlo validation, stability analysis, production freeze, progression synthesis
I	Chapter 7	<code>methylmapper</code>	DMP-to-gene mapping and gene-level aggregation
I	Chapter 8	<code>methylenricher</code>	over-representation analysis, pathway graph clustering, and module scoring
I	Chapter 9	<code>methylalignmentqc</code>	deterministic QC parsing and summarization
I	Chapter 10	all	caveats, legacy remnants, and claims that should be softened
II	Chapter 11	<code>methylutils</code> (ProjectConfig)	anatomy of the project JSON as the single source of truth
II	Chapter 12	all	the two operational workflows with theory, commands, and diagnostics

Part	Chapter	Primary packages	Role
III	Chapter 13	all	exhaustive step_config parameter tables
IV	Chapter 14	all	end-to-end user guide

Part I

Part I: Mathematical and Statistical Foundations

2 MethylUtils

2.1 Scope

`methylutils` is the mathematical spine of the repository. The core implementations live in:

- `methyl_utils/core/centroid_builder.py`
- `methyl_utils/core/distribution_views.py`
- `methyl_utils/methyl_centroid_pair.py`
- `methyl_utils/statistical_tests.py`
- `methyl_utils/ecdf_classifier.py`

Those modules define the empirical-distribution view used by centroid comparison, DMP detection, and ECDF classification. They also expose several auxiliary or legacy methods, especially beta-based distances, that remain available as optional utilities.

2.2 Implemented Method Inventory

Area	Main code path	Status in this book
centroid sufficient statistics	<code>core/centroid_builder.py</code>	principled
ECDF reconstruction from histograms	<code>core/distribution_views.py</code>	principled numerical representation
continuous overlap integral	<code>statistical_tests.py</code>	approximate numerical integration
KS testing on reconstructed ECDFs	<code>statistical_tests.py</code>	asymptotic and grid-based
Mann-Whitney from binned counts	<code>statistical_tests.py</code>	approximate because it reconstructs rank information from histograms
Storey q-values	<code>statistical_tests.py</code>	principled, with standard tuning caveats
DerSimonian-Laird style heterogeneity	<code>statistical_tests.py</code>	principled method-of-moments adaptation
canonical biological effect size	<code>statistical_tests.py</code>	heuristic ranking score
ECDFClassifier	<code>ecdf_classifier.py</code>	principled scoring rule plus heuristic stabilizers
beta distances and overlaps	<code>metrics_core.py</code>	auxiliary; not the main detector-classifier path

2.3 Centroid Sufficient Statistics

For each locus i , the builder stores per-sample methylation fractions x_{si} together with aggregate statistics:

$$\hat{\mu}_i = \frac{S_{x,i}}{N_i}, \quad \hat{\sigma}_i^2 = \max\left(\frac{S_{x^2,i} - S_{x,i}^2/N_i}{\max(N_i - 1, 1)}, \varepsilon\right), \quad (2.1)$$

where N_i is the number of samples contributing to that locus and ε is a small numerical floor.

The centroid builder also stores count-level summaries:

$$S_{m,i} = \sum_s m_{si}, \quad S_{u,i} = \sum_s u_{si}, \quad S_{c^2,i} = \sum_s (m_{si} + u_{si})^2, \quad S_{wx^2,i} = \sum_s \frac{m_{si}^2}{m_{si} + u_{si}}, \quad (2.2)$$

which are later reused by the heterogeneity calculation. Finally, each locus accumulates histogram counts c_{ib} over shared bin edges $\{e_b\}$, producing the `binned_stats` object that drives both ECDF reconstruction and histogram-based testing.

2.4 ECDF Reconstruction

The code constructs a cumulative distribution at bin edges by normalizing cumulative bin counts:

$$\tilde{F}_i(e_b) = \frac{\sum_{r \leq b} c_{ir}}{\sum_{r=1}^B c_{ir}}, \quad b = 0, \dots, B. \quad (2.3)$$

`ECDFView` then interpolates these edge values with a monotone PCHIP spline ([Fritsch and Carlson 1980](#)). The resulting representation supplies:

- a continuous cumulative distribution $\tilde{F}_i(x)$,
- a continuous density proxy $\tilde{f}_i(x) = \max(\partial_x \tilde{F}_i(x), \varepsilon)$, and
- numerically stable means and variances via the stored sufficient statistics.

This is an empirical-distribution model, not a beta or Gaussian fit. That distinction is central to the detector-classifier path.

2.5 Two-Sample Comparison

2.5.1 KS Statistic on Reconstructed ECDFs

For aligned centroid pairs, the code evaluates the two-sample Kolmogorov-Smirnov statistic

$$D_i = \sup_{x \in [0,1]} \left| \tilde{F}_{1i}(x) - \tilde{F}_{2i}(x) \right|. \quad (2.4)$$

Implementation detail matters here: the supremum is approximated on a uniform grid rather than computed analytically, and the p -value is obtained from the large-sample Smirnov distribution ([Kolmogorov 1933](#); [Feller 1948](#)) using the harmonic mean effective sample size

$$n_{\text{eff},i} = \frac{2}{1/\max(n_{1i}, 1) + 1/\max(n_{2i}, 1)}, \quad p_i = \Pr(K \geq \sqrt{n_{\text{eff},i}} D_i). \quad (2.5)$$

This is a reasonable screening approximation, but it is not an exact finite-sample KS test on raw observations.

2.5.2 Mann-Whitney from Histogram Counts

The repository also implements a vectorized Mann-Whitney calculation directly from centroid histograms. Let $c_{ib}^{(1)}$ and $c_{ib}^{(2)}$ be the two groups' bin counts at locus i . The code reconstructs the U statistic as

$$U_i = \sum_{b=1}^B c_{ib}^{(1)} \left(\sum_{r < b} c_{ir}^{(2)} + \frac{1}{2} c_{ib}^{(2)} \right). \quad (2.6)$$

This is followed by the usual normal approximation,

$$\mu_{U,i} = \frac{n_{1i}n_{2i}}{2}, \quad Z_i = \frac{U_i - \mu_{U,i}}{\sqrt{\text{Var}(U_i)}},$$

with a tie correction derived from the combined histogram counts. The important caveat is that the code does **not** reconstruct the full rank ordering of raw methylation values; it reconstructs a rank-like summary from histogram bins. In this book that is treated as an approximation to the classical Mann-Whitney test ([Mann and Whitney 1947](#)), not as an identical procedure.

2.5.3 Heterogeneity

`dl_heterogeneity()` adapts the DerSimonian-Laird method-of-moments estimator ([DerSimonian and Laird 1986](#)) to the centroid summaries:

$$\begin{aligned}\theta_i &= \frac{S_{m,i}}{S_{m,i} + S_{u,i}}, & Q_i &= S_{wx^2,i} - \frac{S_{m,i}^2}{S_{m,i} + S_{u,i}}, \\ \tau_i^2 &= \max\left(0, \frac{Q_i - \max(N_i - 1, 0)}{\max((S_{m,i} + S_{u,i}) - S_{c^2,i}/(S_{m,i} + S_{u,i}), \varepsilon)}\right).\end{aligned}\tag{2.7}$$

This is a useful heterogeneity diagnostic, but it should be described as a method-of-moments adaptation to stored centroid summaries, not as a full random-effects model fit.

2.6 Continuous Overlap

Once densities have been reconstructed, the code measures shared support through

$$O_i = \int_0^1 \min(\tilde{f}_{1i}(x), \tilde{f}_{2i}(x)) \, dx.\tag{2.8}$$

Numerically, the integral is evaluated on a dense grid after each density is renormalized by trapezoidal integration. This makes the overlap robust to small interpolation drift, but it remains a grid approximation.

2.7 Multiple Testing and p-Value Aggregation

For genome-wide DMP screening, `storey_qvalues()` estimates the null fraction π_0 across a grid of λ values and then computes monotone q-values ([Storey 2003](#)):

$$\begin{aligned}\hat{\pi}_0(\lambda) &= \frac{\#\{p_j > \lambda\}}{(1 - \lambda)m}, & \hat{\pi}_0 &= \min(1, \text{median}_\lambda \hat{\pi}_0(\lambda)), \\ q_{(i)} &= \min_{j \geq i} \frac{\hat{\pi}_0 m p_{(j)}}{j}.\end{aligned}\tag{2.9}$$

The same module also provides Fisher ([Fisher 1925](#)) and Stouffer ([Stouffer et al. 1949](#)) style p-value combination utilities. Those are not central to the detector itself, but downstream packages reuse them conceptually for gene-level aggregation.

2.8 Canonical Biological Effect Size

The repository separates statistical significance from a biological ranking score. The implemented canonical effect size is

$$\text{effect}_i = |\Delta\mu_i| (1 - O_i) \exp\left[-\lambda_{\text{var}} \left(\sqrt{\sigma_{1i}^2} + \sqrt{\sigma_{2i}^2}\right)\right]. \quad (2.10)$$

An optional mean-level weight $g(\bar{\mu}_i)$ can be applied, with the code supporting square-root, linear, saturating, and boundary-threshold variants.

This score is useful operationally, but it is not a test statistic, likelihood ratio, or posterior quantity. It is a deliberately engineered ranking rule. Publication text should therefore describe it as a heuristic biological prioritization score.

2.9 ECDFClassifier

The classifier in `ecdf_classifier.py` turns the centroid histograms into per-class densities and scores a sample $x = (x_1, \dots, x_p)$ via a weighted mean log-density:

$$\text{score}_k(x) = \frac{\sum_{i=1}^p w_i \log f_{ki}(x_i)}{\sum_{i=1}^p w_i}. \quad (2.11)$$

The class probabilities are then computed by temperature-scaled softmax:

$$P(Y = k | x) \propto \exp\left(\frac{\text{score}_k(x)}{T_{\text{eff}}}\right), \quad T_{\text{eff}} = T\sqrt{n_{\text{effective}}}, \quad (2.12)$$

where $n_{\text{effective}} = (\sum_i w_i)^2 / \sum_i w_i^2$ is the inverse-Simpson effective number of weighted loci.

Two stabilizers are important because they materially affect behavior:

1. log-densities are floored before averaging, so extremely small density values cannot dominate the score, and
2. unavailable loci are removed from the denominator by setting their effective weights to zero.

The result is a practical Naive Bayes style classifier ([Domingos and Pazzani 1997](#)), but it is not a fully calibrated generative posterior unless the independence and density assumptions actually hold. Optional Platt scaling ([Platt 1999](#)) may further recalibrate the binary probabilities after the ECDF score has been computed.

2.10 Auxiliary Beta-Based Metrics

`metrics_core.py` still contains exact beta divergences, beta overlap surrogates, and moment calculations. The repository also contains a legacy beta-comparator module (`methy1_utils/comparison.py`). These are mathematically legitimate utilities, but they are **not** the main production path for DMP detection plus classification and are not used by the active detector/classifier runtime flow. In publication text, beta distances should therefore be described as an auxiliary branch of the repository, not as the canonical detector-classifier model.

3 MethylCentroid

3.1 Role

`methylcentroid` is the package that turns a cohort of samples into a reusable centroid object. The heavy mathematical lifting is delegated to `methylutils`, but `methylcentroid` fixes the operational semantics of what a centroid means in this repository:

- which samples are aggregated,
- how coverage outliers are handled,
- which sufficient statistics are persisted, and
- how the resulting HDF5 artifacts are organized for downstream steps.

The main implementation lives in `packages/methylcentroid/methyl_centroid/methyl_centroid.py`, with accumulation handled by `methyl_utils/core/centroid_builder.py`.

3.2 What A Centroid Stores

Each persisted centroid contains:

- genomic positions,
- context metadata,
- the sufficient statistics N_i , $S_{x,i}$, $S_{x^2,i}$,
- count summaries $S_{m,i}$, $S_{u,i}$, $S_{c^2,i}$, $S_{wx^2,i}$, and
- per-locus histogram counts used to reconstruct ECDFs.

That is why the centroid is more than a mean profile. It is a compressed empirical-distribution object designed to support both detector-side testing and classifier-side scoring.

3.3 Sample-To-Centroid Update Rule

Given one sample with per-locus methylation fraction

$$x_{si} = \frac{m_{si}}{m_{si} + u_{si}},$$

the centroid builder updates

$$N_i \leftarrow N_i + 1, \quad S_{x,i} \leftarrow S_{x,i} + x_{si}, \quad S_{x^2,i} \leftarrow S_{x^2,i} + x_{si}^2. \quad (3.1)$$

It also updates the count-derived quantities

$$\begin{aligned}
S_{m,i} &\leftarrow S_{m,i} + m_{si}, & S_{u,i} &\leftarrow S_{u,i} + u_{si}, \\
S_{c^2,i} &\leftarrow S_{c^2,i} + (m_{si} + u_{si})^2, & S_{wx^2,i} &\leftarrow S_{wx^2,i} + \frac{m_{si}^2}{m_{si} + u_{si}},
\end{aligned} \tag{3.2}$$

and increments the appropriate methylation histogram bin for x_{si} .

These updates make the centroid a compact empirical summary rather than a full sample-by-locus matrix.

3.4 Coverage Capping By Binomial Thinning

One of the most important preprocessing choices in this package is optional coverage capping. The code implements binomial thinning:

1. for any locus with coverage $n_i = m_i + u_i > n_{\text{cap}}$,
2. compute $p_i = n_{\text{cap}}/n_i$,
3. draw

$$m'_i \sim \text{Binomial}(m_i, p_i), \quad u'_i \sim \text{Binomial}(u_i, p_i). \tag{3.3}$$

This preserves the methylation fraction in expectation while reducing the influence of ultra-deep loci. In other words, it is a variance-control and robustness device, not a biological model.

The implementation exposes both manual capping and auto-capping heuristics. The thinning itself is principled; the choice of n_{cap} is still a workflow decision.

3.5 Auto-Cap Heuristic

The package documentation describes automatic coverage capping through IQR-style heuristics computed from sampled positions. That should be interpreted as an engineering rule:

$$n_{\text{cap}} \approx Q_3 + k \cdot \text{IQR},$$

with implementation-specific sampling and fallbacks. This is useful operationally, but it is not a statistically identified parameter.

3.6 Why Histograms Matter

The centroid builder stores histogram counts even though the mean and variance are already available. That is because the downstream detector and classifier do not rely only on moments. They reconstruct empirical distributions from the binned counts and then use those reconstructed distributions for overlap calculations, KS testing, and classification.

This makes `methylocentroid` a crucial bridge package: it is the step where sample-level methylation is transformed into an empirical-distribution object that later packages can reuse without reloading the full cohort.

3.7 Publication Guidance

For publication text, the safest description is:

- `methylocentroid` constructs empirical per-locus cohort summaries,
- it preserves both moments and histogram information,
- it optionally reduces extreme coverage through binomial thinning, and
- it is upstream of all ECDF-based detector and classifier logic.

The package should **not** be described as fitting a parametric methylation model. It is a summary constructor for the nonparametric empirical-distribution path.

4 MethylDetector

4.1 Role

`methyldetector` turns aligned centroids into a filtered set of differential methylation positions (DMPs) and exports the classifier artifacts used downstream. The main implementation is `packages/methyldetector/methyl_detector/core/methyldetector.py`.

Conceptually, the detector is a two-stage pipeline:

1. statistical screening of loci, and
2. biological prioritization and export.

That split is important. The code does not rely on a single monolithic significance score.

4.2 Alignment And Candidate Set

The detector starts from aligned centroid pairs, typically one control centroid and one disease centroid for a given chromosome and context. The alignment step reduces both centroids to the common set of genomic positions and propagates the empirical summaries described in [Chapter 2](#) and [Chapter 3](#).

Let $\mathcal{J}$ denote the set of aligned loci under consideration for one comparison.

4.3 Statistical Screening

The detector supports two closely related statistical paths:

- a histogram-based path through `MethylCentroidPair.compare_centroids()`, and
- a more precise ECDF path that recomputes KS statistics directly from reconstructed empirical distributions.

In both cases the active workflow is:

1. compute one p-value p_i per locus $i \in \mathcal{J}$,
2. adjust all p-values to q-values using Storey q-values,
3. keep loci satisfying the configured q-value threshold and any minimum effect thresholds.

At the theory level this is

$$\mathcal{D}_{\text{stat}} = \{i \in \mathcal{I} : q_i \leq \alpha, |\Delta\mu_i| \geq \delta_{\text{min}}\}, \quad (4.1)$$

with the exact choice of α and δ_{min} coming from configuration rather than mathematical derivation.

4.4 Biological Ranking

Once a statistically filtered set exists, the detector ranks loci using the canonical effect score introduced in Eq. 2.10. This gives a second set of priorities that is explicitly biological or practical rather than inferential.

The central design choice is that a locus can be:

- statistically strong but biologically modest,
- biologically large but not significant after multiplicity correction, or
- both.

The detector keeps these notions separate instead of compressing them into one scalar.

4.5 Cumulative Effect Coverage

One of the package’s characteristic filters is cumulative effect-mass selection. After sorting retained loci by decreasing effect size, the detector may keep only the smallest prefix whose cumulative effect mass reaches a specified fraction γ of the total:

$$\sum_{i \in S_\gamma} e_i \geq \gamma \sum_{i \in \mathcal{D}_{\text{stat}}} e_i, \quad (4.2)$$

where e_i is the per-locus effect score.

This is a useful way to compress large DMP sets, but it is a heuristic summarization rule rather than a statistical decision boundary with a formal optimality guarantee.

4.6 Context And Chromosome Granularity

The detector is designed to run at chromosome and context granularity. In practical terms this means:

- each classifier artifact is tied to a specific chromosome,
- contexts such as **CG**, **CHG**, and **CHH** can be modeled separately or together, and
- any final multiclass or multi-chromosome system is assembled later by **methylclassifier**.

That architecture keeps detection local and composable, but it also means that downstream fusion is not part of the detector’s statistical test itself.

4.7 Exported Classifier Artifacts

The detector does more than report DMPs. It also packages the information needed by the ECDF classifier:

- the filtered DMP positions,
- per-class centroid histograms over those positions,
- directions of methylation change, and
- feature weights such as effect size.

Mathematically, this means the detector defines the feature space and the feature weighting scheme for the classifier. The classifier is therefore downstream of detector choices even when its own scoring rule is fixed.

4.8 Discovery versus prediction exports

Interpretation workflows (mapping DMPs to genes, pathway enrichment, comparing stages) benefit from **broader, biologically coherent** DMP lists. Predictive modeling benefits from **smaller, stable** feature sets chosen without conflating biological narrative with classification utility.

The implementation therefore supports two coordinated outputs when `dmp_export_mode=dual` (default in project configs):

- `dmps-{chrom}-discovery.csv`: all biologically filtered loci after importance ranking, **without** the effect-size elbow used to trim the classifier panel (unless `discovery_dynamic_dmp_cutoff_enabled` is turned on).
- `dmps-{chrom}-classifier.csv`: the **prediction panel** after the same elbow heuristic as before, or after optional validation balanced-accuracy optimization (`classifier_dmp_selection=featurecuts_` when validation samples are configured).

The saved ECDF classifier pickle always uses the **classifier** panel, not the discovery table. MethylMapper and MethylEnricher should read the **discovery** glob (for example `dmps-*-discovery.csv`). Multiclass merge utilities resolve `dmps-*-classifier.csv` automatically when present.

For strict evaluation of a deployed classifier, feature selection must be performed **inside** each outer training fold (for example by running detection on fold-specific centroids, as in Monte Carlo train splits); discovery lists built on all samples may still be reported for mechanism, but they must not be the sole basis for performance claims unless that design is stated explicitly.

4.9 Production Freeze Panel

In the production workflow (Chapter 12), the detector can bypass statistical and biological discovery entirely and instead operate from a fixed list of pre-selected loci. This is configured via the `fixed_dmp_panel` parameter in `step_config.detection`.

When `fixed_dmp_panel` is set to a path pointing at a CSV file that contains at minimum the columns `chromosome` and `position`, the `_run_multi_context` method short-circuits the entire discovery pipeline. No centroids are compared, no KS statistics are computed, and no biological filtering is applied. The columns `context`, `effect_size`, `statistical_dmp`, and `biological_dmp` are inferred or set to unit defaults for any that are missing from the file. The result object is constructed directly from the loaded rows, with `biological_retention_rate = 1.0` and `n_statistical_dmps = n_biological_dmps = n_classifier_dmps` all equal to the CSV row count.

This mechanism serves the production workflow where:

1. DMP positions were already validated across many Monte Carlo splits (stability analysis),
2. only the most reliably recurring positions are retained in the stable panel, and
3. the final model must use exactly those positions on the full dataset without re-running discovery.

The code path is:

```
MethylDetector._run_multi_context()
↓ if config.fixed_dmp_panel is not None
↓ pd.read_csv(fixed_path)
↓ validate columns {"chromosome", "position"} columns
↓ infer missing context / effect_size / flags
↓ export CSV + save classifier model
↓ return MethylDetectorResult (no discovery performed)
```

The exported classifier model (PKL) is identical in structure to one built through normal discovery; downstream tools (`methylassifier`, `methylassifier`) are unaware of whether the positions came from live detection or a fixed panel.

Theoretical note. Using a fixed panel from a previous analysis is a form of transfer. The panel was selected on a different (possibly overlapping) set of samples. The validity of downstream statistical claims depends on whether the panel samples and the inference samples are from the same population. This is an operational design choice, not a statistical guarantee; it should be stated explicitly in any publication that uses the freeze workflow.

4.10 DMP Coverage in New Samples

A frequently encountered diagnostic problem is low DMP coverage in validation or prediction samples: the classifier reports that only 10–30% of DMP positions have usable observations in a given sample. The root cause is almost always a mismatch between two coverage thresholds:

- `min_coverage` in `step_config.centroid.base_config` — the minimum coverage required for a locus to be considered by centroid construction and DMP detection during training.
- `min_sample_coverage` in `step_config.detection` — the minimum coverage required for a locus to be usable in a new sample during classification.

If centroid construction required `min_coverage = 4` but `min_sample_coverage` is left at its default of 10, any locus with between 4 and 9 reads in a test sample will be dropped. Since typical WGBS data is sequenced at moderate depth and coverage varies across the genome, a large fraction of the 60 000 – 200 000 DMP positions selected during training may fall in this 4–9 read range in test samples, causing the effective feature matrix to collapse to 20% of its intended size.

The observable symptom is:

```
sample_dmp_coverage.dmps_used_fraction_median < 0.5
```

in `validation_metrics.json`. Values below 0.3 indicate severe coverage mismatch and nearly always lead to prediction dominated by the majority class.

Recommended practice: Set `min_sample_coverage` equal to the centroid `min_coverage` used during training. In most projects this means `min_sample_coverage: 4` when `base_config.min_coverage: 4`. A higher `min_sample_coverage` is appropriate only when test samples are known to be sequenced at substantially higher depth than training centroids and locus-level precision is paramount.

4.11 Publication Guidance

The most accurate publication description is:

- `methyldetector` performs nonparametric locus-wise screening,
- applies FDR control through Storey q-values,
- supplements significance with heuristic biological ranking,
- and exports a reduced, weighted empirical-distribution feature set.

When the `fixed_dmp_panel` path is used, the package should be described as applying a pre-specified position set rather than performing novel discovery.

The package should **not** be described as a pure hypothesis-testing engine if the effect-coverage filters are active, because those filters are explicitly post-test prioritization rules.

5 MethylClassifier

5.1 Role

`methylclassifier` is the orchestration layer that applies one or more binary ECDF classifiers to new samples. The mathematical core still lives in `methyl_utils/ecdf_classifier.py`, but `methylclassifier` determines how per-chromosome models, one-vs-rest bundles, and pairwise disease heads are fused into final probabilities.

The relevant implementation files are:

- `packages/methylclassifier/methyl_classifier/core/classifier.py`
- `packages/methylclassifier/methyl_classifier/core/multiclass_ovr.py`

5.2 Binary Classification

For a single binary model, the underlying score is the ECDF log-density average described in Eq. 2.11 and Eq. 2.12. From the perspective of `methylclassifier`, the main practical issues are:

- aligning the input sample to the stored DMP order,
- constructing an availability mask for missing loci, and
- optionally applying a stored Platt calibrator.

If the binary classifier is used alone, this is the cleanest statistical story in the package.

5.3 Multi-Chromosome Binary Classification

When a detector run exports one classifier per chromosome, `methylclassifier` computes chromosome-specific binary probabilities and then fuses them using normalized weights:

$$\hat{p}(y | x) \propto \sum_{c \in \mathcal{C}} w_c \hat{p}_c(y | x_c), \quad \sum_{c \in \mathcal{C}} w_c > 0. \quad (5.1)$$

The code supports several ways to determine the weights w_c :

- fixed configuration weights,
- trimmed mean effect-size weights derived from detector output,
- fitted linear, ridge, lasso, logistic, or elastic-net models trained on validation probabilities.

The last family is best understood as a second-stage meta-model that uses per-chromosome probabilities as features. It can improve prediction, but it is no longer a single coherent generative model.

5.4 One-vs-Rest (OvR) Multiclass Fusion

For multiclass problems the package builds K binary heads and fuses them into a K -class probability vector. The active implementation in `multiclass_ovr.py` uses binary probabilities or logits as intermediate scores and then applies a softmax-like normalization. In symbolic form,

$$\ell_k(x) = \log \hat{p}_k(y = k \mid x) - \log \hat{p}_k(y \neq k \mid x),$$

$$\hat{P}(Y = k \mid x) = \frac{\exp(\ell_k(x))}{\sum_{r=1}^K \exp(\ell_r(x))}. \quad (5.2)$$

This is a practical multiclass reduction. It is not guaranteed to recover a single joint Bayesian posterior unless the binary heads are themselves perfectly compatible.

5.5 Pairwise Control Aggregation

Some project layouts export only pairwise control-vs-disease detectors. In that case the package constructs the control class probability from the geometric mean of the pairwise control probabilities:

$$\hat{p}_{\text{ctrl}}(x) = \exp \left(\frac{1}{K-1} \sum_{k=1}^{K-1} \log \hat{p}_{\text{ctrl}|\text{ctrl-vs-}k}(x) \right). \quad (5.3)$$

The complement $1 - \hat{p}_{\text{ctrl}}(x)$ is then used to form a binary control-vs-rest head before final row normalization.

This is a useful aggregation rule, but it is a heuristic. The geometric mean is not derived in the code from a global multiclass likelihood.

5.6 Bipartite Multi-Control $\times$ Multi-Disease Aggregation

The package also supports bipartite layouts with several control strata and several disease strata. Here it aggregates one probability column across several pairwise experts, again using geometric means. The logic is similar to Eq. 5.3 but now applied to either the control column or the disease column depending on which class head is being built.

That construction is convenient for complex cohort hierarchies, but it should be presented as a carefully engineered reduction strategy rather than a textbook probabilistic model.

5.7 Calibration

When enabled, the package can apply Platt scaling (Platt 1999) to the raw binary outputs. In effect, one learns parameters a and b so that a raw score s is mapped to

$$\hat{p}(y = 1 \mid s) = \frac{1}{1 + \exp(as + b)}. \quad (5.4)$$

This improves probability calibration under distribution shift or class imbalance, but it should not be confused with changing the underlying classifier likelihood.

5.8 New Modeling Features in Production

Recent versions of `methylassifier` expose a broader post-detector modeling layer than the original binary ECDF-only story. In practice, production runs may combine several of the following:

- native multiclass histogram model export (`multiclass-classifier.pkl`) from per-comparison detector outputs,
- optional learned multinomial logistic head on top of histogram pre-softmax scores,
- optional isotonic calibration for probability reshaping,
- optional elastic-net stacking for chromosome-level fusion,
- hierarchical panel readout for family-level reporting.

These are all valid engineering extensions, but conceptually they sit **on top of** the ECDF likelihood core rather than replacing it.

5.8.1 Learned Multiclass Head

The learned head takes base multiclass score vectors $z(x)$ and fits a multinomial logistic layer:

$$\hat{P}(Y = k \mid x) = \frac{\exp(\beta_k^\top z(x) + b_k)}{\sum_{r=1}^K \exp(\beta_r^\top z(x) + b_r)}.$$

This layer is controlled by `multiclass_train_learned_head` and related hyperparameters in `step_config.detection` (see Chapter 13). It is especially useful when the raw histogram scores have good ranking behavior but imperfect class boundaries.

5.8.2 Isotonic and Stacking Layers

Two additional modeling switches in `step_config.classifier` are important for current production behavior:

- `use_isotonic_calibration`: nonparametric monotone calibration of predicted probabilities,
- `use_elasticnet_stacking`: linear meta-model for chromosome-weight fusion under elastic-net regularization.

Both improve empirical calibration or discrimination in many cohorts, but they increase model complexity and should be reported explicitly in methods sections.

5.8.3 Practical Reproducibility

For reproducibility claims, report:

1. whether the learned multiclass head was enabled,
2. whether class balancing was used (`multiclass_learned_class_weight`),
3. whether isotonic calibration and/or elastic-net stacking were enabled,
4. the sklearn version stored in PKL metadata.

Without these details, two runs with the same DMP panel can still produce materially different probabilities.

5.9 Multiclass Learned Head: Class Weighting

When the native multiclass histogram classifier is extended with a learned logistic head (`multiclass_train_learned_head: true`), the logistic regression is fitted on top of the pre-softmax histogram scores from the base model. This second-stage model can accept a `class_weight` parameter (exposed as `multiclass_learned_class_weight` in `step_config.detection`; see Chapter 13).

With `multiclass_learned_class_weight: "balanced"`, sklearn computes inverse-frequency weights:

$$w_k = \frac{n_{\text{total}}}{K \cdot n_k}, \quad (5.5)$$

where n_k is the number of training samples in class k , K is the number of classes, and $n_{\text{total}} = \sum_k n_k$. These weights are passed to the loss function of the multinomial logistic regression, penalizing errors on minority classes more heavily.

This is particularly important for the prostate cancer staging problem where the healthy control cohort may be two to four times larger than any individual disease stage cohort. Without balancing, the logistic head learns to predict the healthy class as the default when the pre-softmax histogram scores do not provide strong class separation (for example, when only 20% of DMP positions are covered in a test sample; see § 4.10).

Theoretical note. Class weighting corrects for imbalance in the empirical class frequencies of the training set. It does not correct for population prevalence differences between training and deployment contexts. If the intended clinical population has very different disease prevalence from the training cohort, additional calibration against the target distribution is required.

Reproducibility. Since sklearn 1.4, the version of sklearn used to train the logistic head is saved in the model PKL metadata under the key `sklearn_version`. This allows detection of version mismatches between training and inference environments. The key is written by `_fit_learned_multiclass_head` in `packages/methylclassifier/methyl_classifier/utils/multiclass_bui`

5.10 Missing Data Semantics

A practical but important detail is how the classifier handles unavailable loci. The implementation masks those loci out of the weighted average rather than imputing them with a full modeled distribution. As a result, the effective evidence for a sample depends on how many DMPs are observed for that sample.

This is a sensible engineering choice, but it means that the posterior confidence is partly a function of data availability as well as class separation.

5.11 Publication Guidance

The strongest defensible summary is:

- the binary head is an ECDF-based Naive Bayes style score with optional calibration,
- multi-chromosome fusion is a weighted ensemble,
- multiclass OvR is a reduction from binary heads to a normalized K -class score,
- pairwise control aggregation is heuristic,
- and fitted chromosome weights are second-stage discriminative models, not part of the original density model.

That framing is more accurate than calling the entire package a single Bayesian multiclass classifier.

6 MethylPredictor, MethylValidation, and MethylDiseaseProgression

6.1 Role

`methylpredictor`, `methylvalidation`, and `methylDiseaseProgression` do not introduce a new generative methylation model. Instead they evaluate, operationalize, and synthesize outputs exported by detector/classifier/mapper/enricher stages.

- `methylpredictor` applies trained classifier artifacts to new cohorts and computes prediction summaries.
- `methylvalidation` creates repeated train/validation splits and aggregates run-level metrics across Monte Carlo style repetitions.
- `methylDiseaseProgression` aggregates per-comparison mapper/enricher outputs into ordered cross-stage progression tables and labels.

6.2 MethylPredictor: Labeled Evaluation

When labels are available, `methylpredictor` computes standard classification metrics using `sklearn.metrics`. For n labeled samples with true labels y_i and predictions $\hat{y}_i$,

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\hat{y}_i = y_i\},$$

$$\text{BalancedAccuracy} = \frac{1}{K} \sum_{k=1}^K \text{Recall}_k. \quad (6.1)$$

For binary classification the package also reports:

$$\text{Sensitivity} = \text{Recall}_{\text{positive}}, \quad \text{Specificity} = \text{Recall}_{\text{negative}}.$$

For multiclass settings it reports per-class precision, recall, F_1 , as well as macro and weighted F_1 summaries.

These are standard descriptive metrics. Their interpretation depends on how the evaluation cohort was assembled rather than on any special estimator inside the package.

6.3 Blind Prediction Summaries

In blind mode, `methylpredictor` cannot estimate discrimination performance because no truth labels are available. Instead it reports:

- predicted class counts,
- mean predicted probability per class, and
- mean Shannon entropy of the per-sample class probabilities.

For one sample with class probabilities $p_1, \dots, p_K$, entropy is

$$H(p) = - \sum_{k=1}^K p_k \log p_k. \quad (6.2)$$

Low entropy corresponds to confident predictions; high entropy corresponds to diffuse class assignments. This is an uncertainty summary, not an accuracy estimate.

6.4 MethylValidation: Split Strategy

`methylvalidation` implements repeated train/validation splitting by cohort. For each cohort with n samples and train fraction ρ , the code computes

$$n_{\text{train}} = \max(1, \lfloor \rho n \rfloor),$$

and then reduces n_{train} to $n - 1$ if necessary so that the validation set is non-empty for $n > 1$.

This is called “stratified” in the code because each cohort is split separately, but it is not identical to the textbook `StratifiedKFold` abstraction. The package performs independent random shuffles within each cohort and then concatenates the resulting class-specific train and validation groups.

6.5 Monte Carlo Evaluation

The validation package repeatedly:

1. generates train and validation cohorts,
2. writes run-specific project files,
3. runs the full pipeline from centroid through predictor, and
4. aggregates scalar metrics across repetitions.

If m_r denotes some metric from run r , the package reports descriptive summaries such as

$$\bar{m} = \frac{1}{R} \sum_{r=1}^R m_r, \quad s_m^2 = \frac{1}{R-1} \sum_{r=1}^R (m_r - \bar{m})^2,$$

along with empirical percentiles.

This is best interpreted as repeated internal validation of the **whole pipeline**, not as a formal confidence interval for an external deployment population.

6.6 What The Validation Layer Does Not Do

The current implementation does not attempt:

- nested hyperparameter tuning,
- exact sampling-theory confidence intervals for each metric,
- dependence-aware aggregation across repeated splits, or
- external-cohort transportability analysis.

Those omissions are not flaws in themselves, but they matter for publication wording. The package measures empirical variation across repeated pipeline runs; it does not prove generalization bounds.

6.7 Stability Analysis

After completing a set of Monte Carlo runs, the `--stability` mode aggregates DMP recurrence across runs. For each discovery DMP d (a (chromosome, position) pair), define the recurrence indicator

$$\mathbf{1}_r(d) = \begin{cases} 1 & \text{if } d \in \mathcal{D}_{\text{disc},r} \\ 0 & \text{otherwise} \end{cases}$$

where $\mathcal{D}_{\text{disc},r}$ is the set of discovery DMPs from run r . When a minimum balanced accuracy threshold $\beta_{\min}$ is configured, only qualifying runs (those with $\widehat{\text{BA}}_r \geq \beta_{\min}$) contribute to the count. Let R^* denote the number of qualifying runs. The estimated recurrence frequency is

$$\hat{f}(d) = \frac{1}{R^*} \sum_{r=1}^{R^*} \mathbf{1}_r(d). \quad (6.3)$$

A DMP is declared **stable** if $\hat{f}(d) \geq \tau$ for a user-specified threshold τ (default 0.7, `stability_dmp_freq`). The output is a CSV file `stable_dmps_production.csv` containing all stable DMPs with their frequencies, sorted by decreasing frequency.

Gene stability is an analogous measure over enricher gene lists: the frequency at which each gene appears in enrichment results across qualifying runs.

Theoretical note. $\hat{f}(d)$ is an empirical frequency, not a formal probability. It depends on the train fraction ρ , the random seed, and the number of iterations R . With small R the estimate is noisy. The threshold τ is a heuristic engineering choice, not derived from any optimality criterion. Higher τ produces smaller, more reproducible panels; lower τ retains more positions at the cost of including split-dependent noise.

6.8 Production Freeze

The production freeze converts the stable DMP panel into a fixed discovery/interpretable production run on the full dataset. The `--freeze` flag calls `freeze_production_model()` in `packages/methylvalidation/methyl_validation/stability.py`, which:

1. Merges one or more per-chromosome stable DMP CSVs into a single genome-wide panel (`stable_dmps_genomewide.csv`).
2. Loads the base project JSON and injects `fixed_dmp_panel` into `step_config.detection` (pointing at the merged CSV).
3. Writes a `production/project.json` with `project_name = "production"` and `output_base` routed to the production directory.
4. Executes `run_pipeline_for_production()`, which runs: `methyl-centroid` → `methyl-detector(fixed panel)` → `methyl-mapper` → `methyl-enricher`.
5. Optionally runs `methyl-disease-progression` when `step_config.progression.enabled=true`.
6. Writes a `production_summary.json` summarizing the run.

Final model fitting is a separate explicit step: `methyl-validation --model` runs `methyl-classifier` → `methyl-predictor` on `production/project.json`.

If configured (`step_config.validation.require_biological_review_for_model=true`), `--model` is blocked until `biological_review_confirmed=true`.

The production project JSON (`production/project.json`) is the artifact that links the frozen model to the `--predictor-only` workflow: it encodes the centroid directories, detection output paths, and classifier paths so that `methyl-predictor` can resolve them without re-running any upstream steps.

6.9 Hybrid Generative Backend (`model_backend=generative_hybrid`)

The `--model` stage now supports a first-class hybrid generative backend in addition to the legacy ECDF path and the tabular sklearn path. This backend is orchestrated by `methylvalidation` and uses the same detector-derived DMP bundle as the other non-ECDF model backends.

6.9.1 Data Contract and Bundle Reuse

The backend does **not** re-derive DMP biology or feature importance. Instead it consumes the `ModelFeatureBundle` generated from detector exports:

- DMP index (`chromosome, position, context`)
- detector-side `effect_size` and `weight` columns
- class labels resolved from project comparisons/groups

This keeps biological weighting aligned with detector outputs and avoids backend-specific feature redefinition.

6.9.2 Modeling Structure

The implemented hybrid backend follows a VAE-style latent-density pattern:

1. encode weighted methylation features into a latent representation z (current implementation uses a linear latent encoder surrogate),
2. fit class-conditional latent densities (currently diagonal Gaussian),
3. compute class posteriors via Bayes-style normalization.

In symbolic form, for classes $k = 1, \dots, K$:

$$p(z \mid y = k) \approx \mathcal{N}(\mu_k, \text{diag}(\sigma_k^2)),$$
$$\hat{P}(y = k \mid z) = \frac{\pi_k p(z \mid y = k)}{\sum_{r=1}^K \pi_r p(z \mid y = r)}.$$

This yields likelihood-based scoring for both binary and multiclass settings while preserving the detector-derived feature index.

6.9.3 Orchestration in `--model`

When `step_config.validation.model_backend="generative_hybrid"`, `run_pipeline_for_model()` executes:

1. **model-bundle**: `build_model_feature_bundle(...)`
2. **generative-train**: `train_generative_model(...)`
3. **generative-predictor**: `predict_generative_model_from_project(...)`

instead of the legacy `methyl-classifier -> methyl-predictor ECDF` chain.

6.9.4 Configuration Surface

The backend is controlled under `step_config.validation`:

- `model_backend`: "generative_hybrid"
- `model_bundle_dir`, `model_weight_column`
- `generative_latent_dim`
- `generative_kl_weight`
- `generative_density_type` (currently `diag_gaussian`)
- `generative_epochs`, `generative_batch_size`, `generative_seed`
- `generative_calibrate`
- `covariates_path`, `covariate_id_column`
- `generative_covariates_strict`

Those fields are part of the same validation config object as Monte Carlo settings, so backend choice is fully reproducible from project JSON.

6.10 MethylDiseaseProgression Synthesis

`methyl-disease-progression` is a post-freeze synthesis step that treats disease groups as an ordered stage sequence and aggregates entity-level signals across stages.

6.10.1 Inputs and Stage Resolution

For each ordered stage, the code reads:

- mapper combined gene table (`all-gene_name-combined.csv` by default),
- enricher pathway table (`enrichment_merged.csv`, fallback `enrichment_top_q0.05.csv`),
- optional module table (`modules_ranked.csv`).

Stage order comes from:

1. CLI `--ordered-comparison-labels` (highest precedence),
2. `step_config.progression.ordered_comparison_labels` (or alias `ordered_disease_groups`),
3. project comparison order.

6.10.2 Entity Scores

The synthesis creates long-format tables for genes, pathways, and modules with (`entity_type`, `entity_id`, `stage_index`, `rank`, `score`).

- **Genes:** score is max of available gene-weight columns per gene (e.g. `total_weight`, `gene_importance`).
- **Pathways:** when q-values exist, score is transformed to

$$\text{score}_{\text{pathway}} = -\log_{10}(\max(q, 10^{-300})),$$

so larger values indicate stronger enrichment. - **Modules:** score is the module score exported by `modules_ranked.csv`.

6.10.3 Progression Labels

For each entity across ordered stages, the package assigns categorical progression labels:

- `stage_specific`, `early_only`, `late_only`,
- `stable_across_stages`,
- `monotonic_up` / `monotonic_down` (based on adjacent score differences over at least three points).

These are deterministic rule-based labels, not fitted trajectory models.

6.10.4 Outputs

The progression step writes:

- `genes_long.csv`
- `pathways_long.csv`
- `modules_long.csv`
- `entities_progression_labels.csv`
- `summary.json`
- optional `report.md` (when `report_md=true`)

This layer is best interpreted as structured biological summarization across stages, not causal disease dynamics inference.

6.11 Two Workflows Summary

MethylValidation supports two primary operational modes, described in full in Chapter 12. A brief summary for theory readers:

Workflow 1 — Model Creation. Repeated Monte Carlo runs with `--stability` estimate DMP recurrence frequencies from centroid+detector iterations. The `--freeze` step then builds production biological outputs on the stable panel (`fixed_dmp_panel`) and may run progression synthesis. The `--model` step then performs final classifier/predictor execution after optional biological gate checks.

Workflow 2 — Model Use for Prediction. The `--predictor-only` flag evaluates the frozen production model on repeated random holdouts without retraining. This workflow answers the question: *what is the empirical distribution of classification performance for the frozen production classifier under random sampling from the available cohort?*

The two workflows share the same split-and-aggregate infrastructure (`split.py`, `project_gen.py`, `pipeline_runner.py`) but differ in which pipeline steps execute per iteration. Workflow 1 iterations run centroid+detector; Workflow 2 iterations run predictor only.

6.12 Publication Guidance

For publication text:

- describe `methylnpredictor` as the evaluation and application layer for trained classifiers,
- reserve discrimination claims for labeled validation cohorts,
- treat blind-mode entropy as an uncertainty diagnostic only,
- describe `methylnvalidation` as repeated split-sample internal validation of the end-to-end workflow,
- report the stability threshold τ and the number of qualifying runs R^* when using `--stability`, and
- state explicitly whether the production model was trained on all samples or on a split-specific subset.

That is both accurate to the code and defensible statistically.

7 MethylMapper

7.1 Role

`methylmapper` converts locus-level detector output into gene- and region-level annotations. It has two distinct mathematical layers:

1. deterministic genomic mapping through bedtools-style interval logic or an external Azure SQL stored procedure, and
2. gene-level aggregation and disease-evidence enrichment.

The relevant code lives primarily in:

- `packages/methylmapper/methyl_mapper/bedtools_mapper.py`
- `packages/methylmapper/methyl_mapper/mapper.py`
- `packages/methylmapper/methyl_mapper/gene_disease_enricher.py`

7.2 Interval Mapping

The bedtools path maps DMP coordinates to genomic features such as promoters, exons, introns, and gene bodies. This part is deterministic interval algebra rather than statistical inference. Feature-level weights may then be attached, for example promoter weights or exon weights, but those weights are biological priors chosen by the workflow rather than estimated model parameters.

The Azure SQL path calls a stored procedure (`spMapDMP2Genes`). Because the stored procedure logic is external to the repository, its exact mapping semantics should be documented as an external dependency rather than as a transparent in-repo algorithm.

7.3 Gene-Level Combined p-Values

For genes with several mapped DMPs, the package computes a signed weighted Stouffer aggregation. Given DMP p-values p_j , weights w_j , and direction signs $s_j \in \{-1, +1\}$,

$$z_j = \Phi^{-1}(1 - p_j/2), \quad \tilde{z}_j = s_j z_j,$$
$$Z_{\text{gene}} = \frac{\sum_j w_j \tilde{z}_j}{\sqrt{\sum_j w_j^2}}, \quad p_{\text{gene}} = 2 [1 - \Phi(|Z_{\text{gene}}|)] . \quad (7.1)$$

The package then applies Storey q-values to the resulting gene-level p-values.

This is a standard meta-analytic construction in form (Stouffer et al. 1949), but its interpretation requires care: mapped DMPs within one gene are correlated along the genome, so the independence assumption behind classical Stouffer aggregation is only approximate.

7.4 Biological Importance Score

`calculate_biological_importance()` combines detector statistics such as `delta_mean`, `overlap`, and `std` into a bounded gene- or region-level importance score. This is not a likelihood or hypothesis test. It is an engineered prioritization score meant to rank biologically interesting genes after mapping.

Publication text should call this what it is: a heuristic importance score that mixes detector-derived quantities and feature weights.

7.5 Stable Gene Set Optimization

The package also contains routines that choose the number of top DMPs required to obtain a stable gene set. These routines are operationally useful, but they are not estimators with standard sampling distributions. Their parameters, step sizes, and stability thresholds should therefore be documented as tuning knobs.

7.6 External Evidence Enrichment

`GeneDiseaseEnricher` queries external services such as DisGeNET (Piñero et al. 2020), Open Targets (Ochoa et al. 2023), and optional Grok-mediated evidence pipelines. This is a valuable biological annotation layer, but it is not in-repo statistical inference. The final evidence depends on:

- service availability,
- service version,
- cached responses,
- threshold profiles such as `strict` or `balanced`, and
- optional LLM-assisted synthesis.

For publication, that layer must be framed as external evidence gathering and summarization rather than as a mathematically identified part of the detector.

7.7 Publication Guidance

The most defensible description is:

- interval mapping is deterministic annotation,
- gene-level p-values are weighted Stouffer aggregates with dependence caveats,
- gene importance is heuristic,
- and disease evidence is external-service-backed annotation.

That is still scientifically useful, but it is not the same kind of object as the upstream ECDF hypothesis tests.

8 MethylEnricher

8.1 Role

`methylenricher` is a downstream interpretation layer. It takes mapped genes or weighted gene lists and organizes them into pathway-level and module-level summaries. The package mixes three kinds of methodology:

- external enrichment analysis through Enrichr ([Chen et al. 2013](#)),
- graph-based pathway grouping,
- heuristic module ranking and disease-prior labeling.

Because of that mixture, this package should be described as a discovery and prioritization layer, not as a single statistical model.

8.2 Over-Representation Analysis

The enrichment step is delegated to Enrichr through `gseapy.enrichr`. In practice this means the repository submits a gene list to an external enrichment engine and then reads back adjusted p-values, odds ratios, and gene memberships for the selected libraries.

The statistical test itself therefore lives outside the repository. The in-repo code is responsible for:

- choosing the input genes,
- selecting the enrichment libraries,
- filtering the returned tables, and
- carrying forward the adjusted p-values into later scoring steps.

Publication text should cite Enrichr ([Chen et al. 2013](#)) and record the exact library set used in the analysis. In MethylEnricher this can be done either by logging an explicit `libraries` list or by logging a named `library_preset` (for example `cancer-extended` for cancer-focused studies).

8.3 Pathway Graph Construction

The package builds a pathway graph in which nodes are pathways and edges represent gene-set overlap. In the simplest case the edge weight is based on Jaccard similarity:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad (8.1)$$

or a closely related overlap score. Community detection is then performed with the Louvain method (Blondel et al. 2008), with connected-components fallbacks if the required package is unavailable.

This is a standard graph heuristic for organizing pathways, but the resulting modules depend materially on the chosen threshold, graph construction rule, and Louvain resolution behavior.

When broad library presets are used (for example `cancer-extended`), module construction can include tens of thousands of enrichment terms. The implementation now supports pre-clustering term reduction so modules can be built from a focused subset:

- q-value gate (`module_cluster_max_q`): keep only terms with adjusted p-value below a threshold,
- per-library cap (`module_cluster_top_terms_per_library`): keep top N terms per library before graph construction.

These controls change the module graph itself, not only the display layer, so they should be reported in methods sections.

8.4 Module Enrichment Score

`compute_module_enrichment_score()` converts a module into a scalar ranking score by combining:

- mean $-\log_{10}(q)$ across pathways,
- number of pathways,
- number of genes,
- mean gene weight.

In normalized form the score is

$$\text{score}_{\text{enrich}} = 0.4 \tilde{m}_{-\log_{10} q} + 0.2 \tilde{n}_{\text{pathways}} + 0.2 \tilde{n}_{\text{genes}} + 0.2 \tilde{m}_{\text{gene-weight}}, \quad (8.2)$$

where each tilded quantity is clipped to $[0, 1]$ after dividing by a hand-chosen scale such as 10, 50, or 200.

This is explicitly heuristic. It is a ranking function, not an enrichment test.

8.5 Disease Relevance Prior

The package also computes a disease-relevance score by overlap with a curated gene set:

$$\text{score}_{\text{disease}} = \frac{|G_{\text{module}} \cap G_{\text{disease}}|}{|G_{\text{module}}|}. \quad (8.3)$$

When no disease set is available the package returns a neutral value of 0.5. By default the code ships with a prostate-cancer-oriented prior gene set, so that default must not be described as disease-agnostic.

The final module score is then

$$\text{score}_{\text{final}} = 0.7 \text{ score}_{\text{enrich}} + 0.3 \text{ score}_{\text{disease}} . \quad (8.4)$$

This is again a heuristic prioritization rule.

8.6 Theme Normalization And Labels

The package includes theme normalization based on substring matching against a JSON mapping file, plus discrete labels such as **High**, **Medium**, and **Low** relevance. Those labels are workflow conveniences. They should not be presented as probabilistic classifications.

8.7 Publication Guidance

The most accurate description is:

- enrichment significance comes from an external ORA engine,
- pathway modules come from graph clustering over pathway overlap,
- module ranking is heuristic,
- and disease relevance is a curated prior overlap rather than a learned posterior.

That combination can be highly useful for interpretation, but it should be framed as an annotation and prioritization pipeline rather than as formal inference.

9 MethylAlignmentQC

9.1 Role

`methylalignmentqc` is the least inferential package in the repository. It parses alignment and duplication metrics produced by external tools such as Picard and Parabricks, then rewrites them into structured JSON for downstream inspection.

The package therefore belongs in the theory book only so that readers do not over-interpret it. Its main functions are deterministic parsing, normalization, and reporting.

9.2 Deterministic Metric Extraction

The parser reads tabular metrics files, histograms, and summary blocks and maps selected fields into a fixed schema. Typical examples include:

- total reads,
- duplicate counts,
- duplication rate,
- related read-group or library statistics.

There is no new statistical estimator here beyond whatever upstream alignment tool already computed.

9.3 Summary Statistics

When the package computes summaries such as duplication rate, it is applying straightforward algebra:

$$\text{duplication rate} = \frac{\text{duplicate reads}}{\text{total reads}}. \tag{9.1}$$

These are useful QC diagnostics, but they do not form a novel methylation model.

9.4 Log Monitoring

The package also contains log-monitoring utilities that infer progress and estimated remaining time by regular-expression matching against subprocess output. Those values are operational heuristics and may change if the upstream log format changes.

9.5 Publication Guidance

In a publication, `methylationqc` should be described as:

- a QC ingestion and normalization layer,
- dependent on upstream Picard or Parabricks metrics,
- deterministic in its parsing logic, and
- not responsible for new statistical inference.

That framing keeps the package in scope without attributing more theory to it than the code actually implements.

10 Limitations and Open Questions

10.1 Why This Chapter Exists

The repository is scientifically useful, but it is not served well by a perfectly tidy story. The code mixes exact formulas, approximations, heuristics, and external-service calls. This chapter records the main caveats that should be understood before publishing claims based on the pipeline.

10.2 The “ECDF-Only Pipeline” Claim Is Too Broad

The centroid-detector-classifier path is indeed ECDF-centered. However:

- `methylnmapper` uses weighted Stouffer aggregation and heuristic importance scores,
- `methylnricher` uses external enrichment plus heuristic module scoring,
- `methylnalignmentqc` is deterministic QC parsing, not ECDF inference.

Therefore the monorepo as a whole should not be marketed as purely ECDF-only. The safer statement is that the **core supervised detection and classification path** is ECDF-centered.

10.3 Dependence Is Real

Several methods rely on assumptions that are standard but imperfect for methylation data:

- loci within one chromosome are correlated,
- mapped DMPs within one gene are correlated,
- binary OvR heads are not guaranteed to be jointly compatible,
- repeated validation splits are not independent external studies.

This does not invalidate the workflow, but it does change how literally one should interpret p-values, q-values, and meta-analytic combinations.

10.4 Approximation Layers

The code intentionally trades exact raw-data procedures for scalable summaries:

- KS statistics are evaluated on a grid and calibrated asymptotically,
- Mann-Whitney is reconstructed from histograms rather than raw ranks,
- ECDF overlap is integrated numerically after spline reconstruction,
- some distance calculations average only over pairwise-overlapping loci.

These approximations should be reported as approximations, not hidden behind textbook method names alone.

10.5 Heuristic Layers

The following components are best described as engineering decisions:

- the canonical effect-size ranking,
- cumulative effect-mass selection,
- geometric-mean control aggregation for pairwise OvR bundles,
- module scoring constants in `methylenricher`,
- stability thresholds for choosing top DMP counts in `methyldmapper`,
- auto-cap rules for coverage trimming.

These are often useful, but their outputs are not calibrated probabilities or test statistics.

10.6 External Dependencies

Some results depend on systems whose internal logic is not fully versioned inside this repository:

- Enrichr,
- DisGeNET,
- Open Targets,
- Azure SQL stored procedures,
- optional Grok/xAI-assisted evidence synthesis.

Any publication using those layers should record service versions, access dates, and cache policies. Otherwise the results are reproducible only in a weak operational sense.

10.7 Legacy Or Experimental Remnants

The repository still contains legacy components, but their status is now clearer:

- `packages/methylutils/comparison.py` contains a beta-based comparator path that is **not** used by the active detector/classifier production flow.
- A few package-local docs can lag behind code evolution between releases.

To avoid methodological confusion, interpret these as **historical/auxiliary artifacts**, not active runtime estimators.

10.7.1 Current safeguards

The current docs/code governance already reduces this risk:

1. The Quarto theory book (`docs/theory`) is the canonical method reference.
2. Production workflow chapters describe the active ECDF-first detector/classifier path and the current model backends explicitly.
3. Legacy modules remain in-repo for compatibility and experimentation, but publication claims should cite only methods documented as active in this book.

10.7.2 Deprecation Status Snapshot

Component / path	Status	Practical guidance
methylutils ECDF core (<code>core/distribution_views.py</code> , <code>ecdf_classifier.py</code> , centroid pair ECDF comparison path)	active	Safe to cite as current detector/classifier mathematical backbone.
methylvalidation model backends (<code>ecdf</code> , <code>tabular_sklearn</code> , <code>generative_hybrid</code>)	active	Safe to cite when backend and key hyperparameters are reported.
methylmapper, methylenricher, methyldiseaseprogression workflow synthesis	active	Safe to cite as downstream interpretation/synthesis layers (with heuristic/external-service caveats).
<code>methyl_utils/comparison.py</code> beta-comparator module	legacy-supported	Keep for compatibility/auxiliary use; do not describe as active production detector path.
Incomplete/retired experimental branches (historical clustering and partial DP-like ideas)	experimental	Do not cite as validated production methodology.

10.8 Publication-Ready Positioning

The strongest defensible manuscript framing is:

1. the core pipeline summarizes methylation into centroids with empirical distributions,
2. it detects DMPs with nonparametric, ECDF-based screening and FDR adjustment,
3. it classifies samples with weighted empirical-distribution likelihoods,
4. and it layers biological annotation and pathway-level interpretation on top.

Anything stronger than that risks overstating what the code actually does.

10.9 What Would Strengthen The Theory Further

The current implementation could support stronger future papers if the repository adds:

- finite-sample benchmarking of histogram-based Mann-Whitney against raw-rank Mann-Whitney,
- calibration studies for the effective-temperature ECDF classifier,
- dependence-aware gene aggregation,
- more explicit modeling of multiclass consistency,
- and clearer retirement or isolation of legacy experimental branches.

Those are natural next steps, but they are not yet the state of the codebase.

Part II

Part II: Workflows and Production

11 The Project Configuration File

11.1 Role and Design Principle

MethylPipeline is controlled by a single JSON configuration file called the **project configuration file** (usually named `project_<name>.json`). Every pipeline step — centroid construction, DMP detection, classification, mapping, enrichment, prediction, and Monte Carlo validation — derives its inputs, outputs, and parameters from this one file.

The governing design principle is **single source of truth**: no piece of information that is already in the project file should be repeated inside any step-specific configuration. Specifically:

- Sample paths are defined once in the top-level `controls/diseases` block and never duplicated in `step_config.predictor`.
- `output_base` and `samples_base_path` are top-level fields inherited by all steps.
- Monte Carlo validation settings are embedded under `step_config.validation`, not in a separate file.

The Pydantic model that validates the project file is `ProjectConfig` in `packages/methylutils/methyl_utils/pip`

11.2 Top-Level Fields

Field	Type	Description
<code>project_name</code>	string	Used as the output directory name under <code>output_base</code> .
<code>laboratory</code>	string	Metadata; appears in centroid H5 attributes.
<code>batch</code>	string	Metadata; appears in centroid H5 attributes.
<code>output_base</code>	string (path)	Root directory for all pipeline outputs.
<code>samples_base_path</code>	string (path)	Base directory for resolving relative sample paths in cohort CSVs.
<code>chromosomes</code>	list of strings	Chromosomes to process (e.g. ["1", "2", ..., "22", "X", "Y"]).
<code>contexts</code>	list of strings	Methylation contexts: "CG", "CHG", "CHH".

Field	Type	Description
<code>comparisons</code>	string or list	Comparison layout: "control_vs_each_disease" or a list of explicit comparison pairs.

11.3 The Cohort Hierarchy

The `controls` and `diseases` blocks define the study cohort structure. Each block has a `label` (used in output directory naming) and a `groups` list. For hierarchical multiclass designs, disease groups can have `stages`:

```
{
  "controls": {
    "label": "healthy",
    "groups": [
      { "label": "all", "sample_paths": ["configs/healthy.csv"] }
    ]
  },
  "diseases": {
    "label": "cancer",
    "groups": [
      {
        "label": "pca",
        "stages": [
          { "label": "pca1", "sample_paths": ["configs/pca1.csv"] },
          { "label": "pca2", "sample_paths": ["configs/pca2.csv"] }
        ]
      }
    ]
  }
}
```

In this layout, `methy1-centroid` builds one centroid per leaf group (`all`, `pca1`, `pca2`), and `methy1-detector` compares `all` versus each disease stage. The resolved group labels used by the Monte Carlo and predictor subsystems are the concatenated parent-and-leaf names: `pca_pca1`, `pca_pca2`, etc.

For flat two-class binary projects (one control group, one disease group, no stages), the structure simplifies to:

```
{
  "controls": { "label": "healthy", "groups": [{ "label": "all", "sample_paths": ["..."] }] },
  "diseases": { "label": "cancer", "groups": [{ "label": "all", "sample_paths": ["..."] }] }
}
```

11.4 The `step_config` Dictionary

`step_config` is a JSON object whose keys correspond to pipeline steps. Each value is a free-form dictionary of parameters that override the step's defaults. The recognized keys are:

Key	Handled by	Description
<code>centroid</code>	<code>methy1-centroid</code>	Coverage thresholds, histogram bins, GPU, parallelism.
<code>detection</code>	<code>methy1-detector</code>	Statistical test, effect size, DMP selection, classifier export.
<code>mapper</code>	<code>methy1-mapper</code>	GTF, gene window sizes, enrichment source.
<code>enricher</code>	<code>methy1-enricher</code>	Library selection, disease filters, module scoring.
<code>classifier</code>	<code>methy1-classifier</code>	Chromosome weighting, OvR layout, panel definition.
<code>predictor</code>	<code>methy1-predictor</code>	Panel definition, debug mode.
<code>validation</code>	<code>methy1-validation</code>	Monte Carlo settings embedded in the project.
<code>cluster</code>	<code>methy1-cluster</code>	Clustering method, distance metric, k selection.
<code>alignment_qc</code>	<code>methy1-alignment-qc</code>	Schema validation, group filtering.

The exhaustive list of parameters for each key is in [Chapter 13](#).

11.5 The `step_config.predictor` Anti-Pattern

A common mistake when creating project configurations is to copy the `controls/diseases` block from the top level into `step_config.predictor`. This creates duplication: the sample paths are now defined in two places. If the top-level cohort CSVs change, the predictor block must also be updated, creating a maintenance burden and a potential source of inconsistency.

Correct pattern: `step_config.predictor` should contain only predictor-specific overrides — typically `debug`, `panel`, and optionally `multiclass_model_path`. The sample paths are resolved automatically from the top-level `controls/diseases` block during project loading.

Anti-pattern (avoid):

```
"predictor": {  
  "controls": { ... },    ← duplicates top-level controls  
  "diseases": { ... },    ← duplicates top-level diseases  
  "panel": { ... }  
}
```

Correct pattern:

```
"predictor": {
  "debug": false,
  "panel": { "primary_family": "prostate", "families": { ... }, "indeterminate_delta": 0.25 }
}
```

11.6 Embedding Monte Carlo Settings: `step_config.validation`

Rather than maintaining a separate `monte_carlo_config.json` file, the Monte Carlo validation settings can be embedded directly inside the project file under `step_config.validation`. When `methy1-validation` is invoked with `--project <path>`, it reads the `step_config.validation` block and constructs a `MonteCarloConfig` from it, inheriting `samples_base_path`, `output_base`, and the cohort structure from the top-level project fields.

The fields that are appropriate for `step_config.validation` are only the Monte Carlo-specific ones:

```
"validation": {
  "train_fraction": 0.8,
  "n_iterations": 50,
  "seed": 42,
  "run_stability": true,
  "stability_dmp_freq": 0.7,
  "stability_min_balanced_accuracy": 0.6,
  "abort_on_step_failure": false
}
```

Fields that are **not** needed here (because they are inherited from the project): `samples_base_path`, `output_base`, `base_project`, `cohorts`, `healthy_csv`, `disease_csv`.

11.6.1 Modeling backend inside `step_config.validation`

The same `validation` block also controls what happens during `methy1-validation --model`. The default backend is `ecdf` (classifier + predictor), but the project may select:

- `model_backend`: `"tabular_sklearn"` for tabular bundle-based training/evaluation,
- `model_backend`: `"generative_hybrid"` for the hybrid latent-density generative backend.

This keeps backend choice versioned in the same project JSON used for stability/freeze/model orchestration.

11.7 Annotated Example

The following is an annotated excerpt from `configs/project_Healthy_vs_PCa1-4-CG.json`, the reference project for prostate cancer staging:

```
{
  "project_name": "Healthy_vs_PCa1-4-CG",      ← output under output_base/Healthy_vs_PCa1-4-CG,
  "laboratory": "psomagen",
  "batch": "Second batch of prostate cancer samples",
  "output_base": "/work/prostate-cancer",      ← all outputs go here
  "samples_base_path": "/work/prostate-cancer/samples",
  "chromosomes": ["1", "2", ..., "22", "X", "Y"],
  "contexts": ["CG"],
  "comparisons": "control_vs_each_disease",    ← healthy vs each pca stage

  "controls": {
    "label": "healthy",
    "groups": [{ "label": "all", "sample_paths": ["configs/healthy.csv"] }]
  },
  "diseases": {
    "label": "cancer",
    "groups": [{
      "label": "pca",
      "stages": [
        { "label": "pca1", "sample_paths": ["configs/pca1.csv"] },
        { "label": "pca2", "sample_paths": ["configs/pca2.csv"] },
        { "label": "pca3", "sample_paths": ["configs/pca3.csv"] },
        { "label": "pca4", "sample_paths": ["configs/pca4.csv"] }
      ]
    }]
  },
  "step_config": {
    "centroid": {
      "base_config": {
        "min_coverage": 4,      ← positions with < 4 reads excluded from centroid
        "binned_stats_bins": 20, ← 20-bin ECDF histogram stored per locus
        "use_gpu": true
      }
    },
    "detection": {
      "alpha": 0.05,           ← q-value threshold for statistical DMPs
      "min_coverage": 4,       ← same as centroid min_coverage (critical: must match)
      "min_sample_coverage": 4, ← coverage required in new samples (must match centroid)
      "effect_size_coverage": 0.95, ← retain top 95% effect mass
      "multiclass_train_learned_head": true,
      "multiclass_learned_class_weight": "balanced" ← corrects class imbalance
    }
  }
}
```

```

},
"classifier": {
  "ovr_binary_pickles_from_comparisons": true,
  "panel": {
    "primary_family": "prostate",
    "families": { "prostate": ["pca_pca1", "pca_pca2", "pca_pca3", "pca_pca4"] },
    "indeterminate_delta": 0.25
  }
},
"predictor": {
  "debug": false,
  "panel": {           ← same panel as classifier; only override is debug flag
    "primary_family": "prostate",
    "families": { "prostate": ["pca_pca1", "pca_pca2", "pca_pca3", "pca_pca4"] },
    "indeterminate_delta": 0.25
  }
},
"validation": {       ← embedded MC settings; no separate config file needed
  "train_fraction": 0.8,
  "n_iterations": 5,
  "seed": 42,
  "run_stability": true,
  "stability_dmp_freq": 0.6
}
}
}

```

11.8 Output Directory Layout

Given `output_base = "/work/prostate-cancer"` and `project_name = "Healthy_vs_PCa1-4-CG"`, the top-level output structure is:

```

/work/prostate-cancer/
  Healthy_vs_PCa1-4-CG/
    centroids/
      controls/healthy/all/<chrom>-CG.h5
      diseases/cancer/pca_pca1/<chrom>-CG.h5    (and pca2, pca3, pca4)
    detections/
      all/<comparison>/dmps-<chrom>-discovery.csv
      dmps-<chrom>-classifier.csv
      classifier-<chrom>-CG.pkl
    classifiers/
      all/
        pca_pca1/run_0001-classifier.pkl
        multiclass-classifier.pkl

```

```

    classifier_all_run_0001.pkl
mapper/
enricher/
monte_carlo_runs/
    run_0001/
        project.json
        centroids/
        detections/
        logs/
    stability/
        stable_dmps_production.csv
production/
    project.json          ← contains fixed_dmp_panel
    centroids/
    detections/
    mapper/
    enricher/
    progression/          ← optional when step_config.progression.enabled=true
    classifiers/          ← created by --model
    predictors/           ← created by --model

```

12 The Two Workflows

12.1 Overview

MethylPipeline supports two primary end-to-end operational workflows. They are designed to be run sequentially: Workflow 1 produces the final production model; Workflow 2 evaluates that model on random holdouts.

	Workflow 1	Workflow 2
Goal	Build a stable, production-ready model	Evaluate the frozen production model
Flag	<code>--stability</code> , then <code>--freeze</code> , then <code>--model</code>	<code>--predictor-only</code>
Steps run per iteration	centroid → detector	predictor only
Steps run once (freeze/model)	freeze/analysis: centroid → detector(fixed) → mapper → enricher (optional progression); model: classifier → predictor	—
Output	stable panel, production biological outputs, final classifier/predictor outputs	<code>all_metrics.csv</code> , <code>metrics_summary.json</code>
Answers	Which positions and model best separate the classes on all data?	What performance distribution does the frozen model achieve?

Both workflows are controlled entirely by the project configuration file and the `step_config.validation` embedded settings (see Chapter 11).

12.2 Workflow 1: Model Creation

12.2.1 Theoretical Motivation

A naive approach to building a classifier would be to detect DMPs on all available samples, then evaluate the same classifier on held-out subsets. This conflates feature selection with performance estimation: the DMP panel was implicitly chosen to fit all samples, making any leave-out evaluation optimistic.

MethylValidation addresses this by repeating only the discovery portion of the pipeline on random train/validation splits. Each iteration rebuilds centroids and re-runs detection on the training cohort only. The resulting detector-side balanced accuracy summaries are used as internal estimates of discovery robustness and to filter stability runs when requested.

After many such iterations, a DMP that appears consistently in the discovery sets of most runs is more likely to reflect a true biological signal than one that appears in only a few. This motivates the stability analysis: the stable panel is a consensus over repeated random partitions of the data.

The production freeze then trains a final classifier on **all available data** restricted to the stable panel. Because the panel itself was selected using the full sample set (via the MC iterations), this is still an internal analysis; it is not a substitute for external validation on an independent cohort.

12.2.2 The Three Stages of Workflow 1

Workflow 1 is easiest to reason about as three explicit stages with different statistical roles.

12.2.2.1 Stage A — Stability (`--stability`)

Theoretical role. This stage estimates *feature-selection stability* rather than final predictive performance. Let $\mathcal{D}_{\text{disc},r}$ be the discovery DMP set from split r . A locus-level recurrence estimator

$$\hat{f}(d) = \frac{1}{R^*} \sum_{r=1}^{R^*} \mathbf{1}[d \in \mathcal{D}_{\text{disc},r}]$$

acts as a Monte Carlo approximation to selection probability under re-sampling. Thresholding with $\hat{f}(d) \geq \tau$ (where $\tau = \text{stability_dmp_freq}$) enforces reproducibility pressure on the panel.

Practical role. This stage answers: “Which loci survive repeated train/validation perturbations?” In practice:

- it runs only `centroid -> detector` per split (fast enough for many iterations),
- it creates `all_metrics.csv/metrics_summary.json` for detector-side internal diagnostics,
- it writes `stability/stable_dmps_production.csv`, which is the only accepted panel input for `--freeze`,
- and it allows optional run-quality filtering via `stability_min_balanced_accuracy` before recurrence counting.

For most projects, this is the stage where panel size and robustness are tuned (`n_iterations`, `train_fraction`, `stability_dmp_freq`).

12.2.2.2 Stage B — Freeze/Analysis (`--freeze`)

Theoretical role. This stage converts the consensus panel into a deterministic production artifact. The key operation is a *panel-constrained rerun on all samples*: detector inference is no longer discovering a new panel; it is applying fixed loci and producing full downstream biological interpretation from a stable feature space.

Practical role. This stage answers: “Given a locked panel, what is the complete biological analysis on the full cohort?” In practice it:

1. injects `fixed_dmp_panel` into `production/project.json`,
2. rebuilds centroids on all data,
3. runs detector in fixed-panel mode,
4. runs mapper and enricher to produce gene/pathway outputs,
5. optionally runs progression synthesis for ordered disease-stage summaries.

The freeze artifacts (`production/project.json`, fixed-panel detection outputs, mapper/enricher summaries) become the canonical handoff package for model release and review.

12.2.2.3 Stage C — Modeling (`--model`)

Theoretical role. This stage fits the final prediction function conditioned on the frozen panel and frozen production project. It separates *feature agreement* (handled in Stage A/B) from *final classifier parameter estimation* (handled here).

Practical role. This stage answers: “What trained model artifacts should be deployed?” In practice it uses `step_config.validation.model_backend`:

- `ecdf` (default): runs `methy1-classifier` then `methy1-predictor`,
- `tabular_sklearn`: runs model bundle -> tabular training -> tabular predictor evaluation,
- `generative_hybrid`: runs model bundle -> latent-density generative training -> generative predictor evaluation.

For all backends, the optional biological gate (`require_biological_review_for_model`) is enforced before model creation.

The resulting files under `production/classifiers/` and `production/predictors/` are the deployable model artifacts. Any rerun with changed panel or freeze inputs should be treated as a new model version.

12.2.3 Steps

Step 1: Prepare the project configuration

Ensure the project file has `step_config.validation` with at least `train_fraction`, `n_iterations`, `seed`, and `run_stability: true`:

```
"validation": {
  "train_fraction": 0.8,
  "n_iterations": 50,
  "seed": 42,
  "run_stability": true,
  "stability_dmp_freq": 0.7,
  "stability_min_balanced_accuracy": null,
  "abort_on_step_failure": false
}
```

Step 2: Run Monte Carlo with stability

```
source .venv/bin/activate
methyl-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --stability
```

This runs `n_iterations` stratified train/validation splits. Per iteration:

1. `methyl-centroid --group group1` and `--group group2` build separate centroid H5 files for the training split.
2. `methyl-detector --project` detects DMPs between the training centroids.

After all iterations, stability analysis counts DMP recurrence across the discovery sets:

$$\hat{f}(d) = \frac{1}{R^*} \sum_{r=1}^{R^*} \mathbf{1}[d \in \mathcal{D}_{\text{disc},r}]$$

and writes `monte_carlo_runs/stability/stable_dmps_production.csv`.

Step 3: Run the production freeze

```
methyl-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --freeze
```

This calls `freeze_production_model()` and then (optionally) progression synthesis which:

1. Merges stable DMP CSVs into `monte_carlo_runs/production/stable_dmps_genomewide.csv`.
2. Creates `monte_carlo_runs/production/project.json` with `fixed_dmp_panel` injected into `step_config.detection`.
3. Runs production freeze on **all samples**: `centroid` → `detector(fixed panel)` → `mapper` → `enricher`.
4. If `step_config.progression.enabled=true`, runs `methyl-disease-progression` to aggregate cross-stage mapper/enricher outputs.
5. Writes `monte_carlo_runs/production/production_summary.json`.

Step 4: Run final model build

```
methyl-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --model
```

This runs `run_pipeline_for_model()`:

1. methyl-classifier --project production/project.json
2. methyl-predictor --project production/project.json

If `step_config.validation.require_biological_review_for_model=true`, `--model` is blocked until `biological_review_confirmed=true`.

12.2.4 Workflow 1 Diagram

```
%| fig-cap: "Workflow 1: Model Creation - from Monte Carlo splits to production freeze."
\usetikzlibrary{arrows.meta, positioning, shapes.geometric}
\begin{tikzpicture}[
  node distance=0.55cm and 0cm,
  box/.style={draw, rectangle, rounded corners=2pt, text width=6.5cm,
    align=center, font=\small, minimum height=0.65cm, fill=gray!8},
  arr/.style={-{Latex[length=2mm]}}, thick}
]
\node[box] (A) {\texttt{project.json} (single unified config)};
\node[box, below=of A] (B) {\texttt{--stability}: $\mathbb{R}$ full-pipeline iterations};
\node[box, below=of B] (C) {Per iteration: centroid $\to$ detector};
\node[box, below=of C] (D) {Detector metrics per iteration};
\node[box, below=of D] (E) {Stability: compute $\hat{f}(d)$ across runs};
\node[box, below=of E] (F) {\texttt{stable\_dmps\_production.csv}};
\node[box, below=of F] (G) {\texttt{--freeze}};
\node[box, below=of G] (H) {Merge $\to$ \texttt{stable\_dmps\_genomewide.csv}};
\node[box, below=of H] (I) {\texttt{production/project.json} with \texttt{fixed\_dmp\_panel}};
\node[box, below=of I] (J) {Freeze on \emph{all} samples: centroid $\to$ detector(fixed) $\to$};
\node[box, below=of J] (K) {Optional: disease progression synthesis};
\node[box, below=of K] (L) {\texttt{--model}: classifier $\to$ predictor (biological gate opti
\draw[arr] (A)--(B); \draw[arr] (B)--(C); \draw[arr] (C)--(D);
\draw[arr] (D)--(E); \draw[arr] (E)--(F); \draw[arr] (F)--(G);
\draw[arr] (G)--(H); \draw[arr] (H)--(I); \draw[arr] (I)--(J);
\draw[arr] (J)--(K); \draw[arr] (K)--(L);
\end{tikzpicture}
```

12.2.5 Expected Outputs

After Workflow 1 completes:

```
monte_carlo_runs/
  run_0001/ ... run_000N/          ← per-iteration outputs
    detections/**/result*.json
  all_metrics.csv                  ← one row per iteration
  metrics_summary.json             ← mean, std, percentiles per metric
  stability/
    stable_dmps_production.csv
```

```

    stability_summary.json
production/
  project.json          ← frozen project with fixed_dmp_panel
  centroids/           ← built on all samples
  detections/          ← using fixed DMP panel
  mapper/
  enricher/
  progression/         ← optional (when enabled)
  classifiers/         ← created by --model
  predictors/          ← created by --model

```

12.3 Workflow 2: Model Use for Prediction

12.3.1 Theoretical Motivation

Once a production model exists, a natural question is: what balanced accuracy does it achieve on random holdouts drawn from the available cohort? This is not the same as the BA measured in Workflow 1, because Workflow 1 re-trained the model on each split while Workflow 2 applies the frozen model.

Workflow 2 provides the empirical distribution of the frozen model's performance under random sampling. It is faster than Workflow 1 (only the predictor runs per iteration), and it answers the question: *is the production model's accuracy stable across different random splits, or does it depend heavily on which specific samples are held out?*

Important caveat. The available cohort used in Workflow 2 overlaps with the training data for the production model (the production model was trained on all samples). This means Workflow 2 is not a hold-out evaluation in the classical sense; the test samples may also be in the training centroid. A truly external validation cohort would be needed to measure generalization to unseen data.

12.3.2 Steps

Step 1: Confirm the production model exists

The file `monte_carlo_runs/production/project.json` must exist (created by Workflow 1's `--freeze` step).

Step 2: Run predictor-only evaluation

```
methy1-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --predictor-only
```

Per iteration, this:

1. Creates a new split-specific `project.json` with train/validation CSVs.

2. Merges the production model paths from `production/project.json` into the run-specific project (via `apply_frozen_pipeline_artifacts_to_run_project()`).
3. Runs only `methy1-predictor` on the validation split.
4. Writes `validation_metrics.json`.

After all iterations, the same aggregation as Workflow 1 produces `all_metrics.csv` and `metrics_summary.json`.

12.3.3 Workflow 2 Diagram

```
%| fig-cap: "Workflow 2: Model Use for Prediction - predictor-only evaluation of the frozen m
\usetikzlibrary{arrows.meta, positioning, shapes.geometric}
\begin{tikzpicture}[
  node distance=0.55cm and 2cm,
  box/.style={draw, rectangle, rounded corners=2pt, text width=6.5cm,
    align=center, font=\small, minimum height=0.65cm, fill=gray!8},
  arr/.style={-{Latex[length=2mm]}}, thick}
]
\node[box] (A) {\texttt{production/project.json} (frozen model)};
\node[box, below=of A] (B) {\texttt{project.json} (cohorts + validation settings)};
\node[box, below=of B] (C) {\texttt{--predictor-only}: $R$ iterations, predictor only};
\node[box, below=of C] (D) {Per iteration: split $\to$ merge frozen paths $\to$ predictor};
\node[box, below=of D] (E) {\texttt{validation\_metrics.json} per iteration};
\node[box, below=of E] (F) {\texttt{all\_metrics.csv} + \texttt{metrics\_summary.json}};
\draw[arr] (A)--(B); \draw[arr] (B)--(C); \draw[arr] (C)--(D);
\draw[arr] (D)--(E); \draw[arr] (E)--(F);
\end{tikzpicture}
```

12.4 Diagnostic: DMP Coverage

Symptom. `sample_dmp_coverage.dmps_used_fraction_median < 0.5` in `validation_metrics.json`. Values below 0.3 indicate severe degradation.

Cause. The centroid `min_coverage` (e.g. 4) and the detector `min_sample_coverage` (default 10) are mismatched. DMP positions that were detectable in training centroids at 4× coverage become invisible in test samples when the classifier requires 10× coverage.

Fix. In `step_config.detection`, add:

```
"min_sample_coverage": 4
```

This aligns the coverage requirement for classification with the centroid build threshold.

Verification. Re-run Workflow 2. `dmps_used_fraction_median` should rise to 0.6–0.9 for typical WGBS data.

12.5 Diagnostic: Class Imbalance

Symptom. The confusion matrix shows near-perfect recall for the majority class (e.g. healthy/control) and near-zero recall for minority classes (disease stages). Balanced accuracy is close to $1/K$ (random chance).

Cause. The training cohort has substantially more samples in one class. For example, 86 healthy vs 25–36 disease samples per stage (41% healthy in the example project). The multinomial logistic regression learned head defaults to the majority class when the histogram scores are ambiguous (which is common when DMP coverage is low).

Fix. In `step_config.detection`, add:

```
"multiclass_train_learned_head": true,  
"multiclass_learned_class_weight": "balanced"
```

The `balanced` setting computes $w_k = n_{\text{total}} / (K \cdot n_k)$, penalizing majority-class errors proportionally less than minority-class errors. See § 5.9 for the full derivation.

12.6 Diagnostic: Sklearn Version Mismatch

Symptom. Warning messages during prediction: `InconsistentVersionWarning: Trying to unpickle estimator LogisticRegression from version X.Y.Z when using version A.B.C.`

Cause. The `multiclass-classifier.pkl` was serialized with a different version of scikit-learn than the one currently installed.

Diagnosis. Check the `sklearn_version` field in the model PKL metadata:

```
import pickle  
with open("monte_carlo_runs/production/classifiers/multiclass-classifier.pkl", "rb") as f:  
    model = pickle.load(f)  
print(model["metadata"].get("sklearn_version")) # version used at training time
```

Compare to the current installed version:

```
import sklearn; print(sklearn.__version__)
```

Fix. Re-run the `--freeze` step with the current sklearn version. The logistic regression head will be retrained and the new PKL will record the correct version. New model PKLs produced after the fix always record `sklearn_version` in their metadata.

12.7 Choosing Workflow Parameters

12.7.1 `train_fraction`

Higher values (0.8–0.9) leave less data for validation but produce more powerful centroids in training. For small cohorts (total $n < 50$ per class), prefer 0.8. For large cohorts, 0.9 is reasonable.

12.7.2 `n_iterations`

More iterations reduce the variance of the empirical performance summaries. 20–50 iterations is typically sufficient to characterize the mean and standard deviation of balanced accuracy. 100+ iterations are useful for stable percentile estimates (e.g. p5, p95).

12.7.3 `stability_dmp_freq`

Higher values (0.8–1.0) produce smaller, more reproducible panels but may miss positions that are biologically informative but variable across splits. Lower values (0.5–0.7) retain more positions at the cost of including split-sensitive noise. A good starting point is 0.7.

12.7.4 `stability_min_balanced_accuracy`

Setting this parameter (e.g. to 0.6) excludes iterations with poor validation performance from the DMP frequency count. This can improve panel quality by excluding outlier runs where the centroid construction or detection failed to find informative DMPs. It also reduces the effective R^* and thus increases the variance of $\hat{f}(d)$. Use with caution for small `n_iterations`.

Part III

Part III: Configuration Reference

13 Configuration Reference

This chapter provides exhaustive tables of every parameter in every `step_config` section. For theoretical background on each parameter group, see the cross-references given in each section header.

Parameters are listed with their JSON/Python name, type, default value, constraints, and a description. “Required” in the Default column means the field must be present; there is no default.

13.1 `step_config.centroid`

See Chapter 3 for theoretical background.

13.1.1 Batch wrapper fields

Parameter	Type	Default	Constraints	Description
<code>chromosomes</code>	<code>list[str]</code>	(from project)	each 1–22, X, Y	Chromosomes to process in this batch run.
<code>contexts</code>	<code>list[str]</code>	<code>["CG"]</code>	each CG, CHG, CHH	Methylation contexts to process.
<code>parallel_combinations</code>	<code>int</code>	1	1	Number of chromosome×context combinations to run in parallel.
<code>continue_on_error</code>	<code>bool</code>	<code>true</code>	—	If <code>true</code> , log errors and continue; if <code>false</code> , abort on first failure.
<code>save_batch_summary</code>	<code>bool</code>	<code>true</code>	—	Write a JSON summary of the batch run.

13.1.2 base_config fields (MethylCentroidConfig)

Parameter	Type	Default	Constraints	Description
min_coverage	int	4	1	Minimum total (mC + uC) reads per position per sample. Positions below this threshold are excluded from the centroid. Must match <code>step_config.detection.min_c</code> and <code>step_config.detection.min_s</code>
use_gpu	bool	true	—	Use GPU acceleration when available.
max_sample_workers	int null	null	1 or null	Maximum parallel worker processes for sample loading. null uses all available CPUs.
verbose	bool	true	—	Verbose logging during centroid construction.
binned_stats_bins	int	20	1	Number of histogram bins for the per-locus ECDF summary stored in the H5 centroid file. Used by methyl-detector for KS/overlap statistics.
cap_coverage	bool	false	—	If true , apply binomial thinning to cap extreme coverage values before centroid construction.

Parameter	Type	Default	Constraints	Description
<code>cap_coverage_n_cap</code>	<code>int null</code>	<code>null</code>	1 or null	Hard coverage cap. If <code>null</code> and <code>cap_coverage_auto_n_cap</code> is <code>true</code> , the cap is estimated automatically.
<code>cap_coverage_seed</code>	<code>int</code>	0	—	RNG seed for binomial thinning.
<code>cap_coverage_auto_n_cap</code>	<code>bool</code>	<code>false</code>	—	If <code>true</code> , estimate the coverage cap from the first sample using an IQR-based method.
<code>cap_coverage_n_cap_method</code>	<code>str</code>	"iqr"	"iqr"	Method for auto-estimating the coverage cap. Currently only IQR is supported.
<code>cap_coverage_n_cap_float_iqr_multiplier</code>	<code>float</code>	3.0	> 0	IQR multiplier for auto-cap: $\text{cap} = Q3 + \text{multiplier} \times \text{IQR}$.
<code>cap_coverage_n_cap_max_positions</code>	<code>int null</code>	<code>null</code>	1 or null	Maximum positions to sample when auto-estimating the coverage cap. <code>null</code> uses all positions.
<code>laboratory</code>	<code>str</code>	" "	—	Metadata label written to H5 attributes.
<code>disease</code>	<code>str</code>	" "	—	Metadata label written to H5 attributes.
<code>group</code>	<code>str</code>	" "	—	Metadata label written to H5 attributes.

batch	str	""	—	Metadata label written to H5 attributes (usually inherited from project batch).
-------	-----	----	---	---

13.2 step_config.detection — Runtime Parameters

See Chapter 4 for theoretical background. The parameters marked with an asterisk () have direct theoretical counterparts in the chapter.

13.2.1 Input / Output

Parameter	Type	Default	Constraints	Description
fixed_dmp_panel	str (path) null	null	File must exist; must have chromosome and position columns	Path to a fixed DMP panel CSV. If set, bypasses statistical and biological discovery entirely. See § 4.9.
output_dir	str null	(from project)	—	Override for the detection output directory. Usually resolved from the project.

13.2.2 Statistical Screening

Parameter	Type	Default	Constraints	Description
alpha	float	0.05	[0, 1]	q-value threshold for the statistical DMP set $\mathcal{D}_{\text{stat}}$ (Eq. 4.1).

Parameter	Type	Default	Constraints	Description
<code>significance_test</code>	<code>str</code>	<code>"ks_ecdf"</code>	<code>"ks_ecdf"</code> or <code>"mann_whitney"</code>	Statistical test. <code>ks_ecdf</code> uses the KS statistic on reconstructed ECDFs (Chapter 2); <code>mann_whitney</code> uses the histogram-based rank sum approximation.
<code>ecdf_grid_size</code>	<code>int</code>	256	[16, 4096]	Grid resolution for ECDF reconstruction, KS statistics, and overlap integration. Higher values improve accuracy at the cost of compute. Legacy alias: <code>ecdf_overlap_grid_size</code> .
<code>min_coverage</code>	<code>int</code>	4	1	Minimum centroid coverage required for a locus to be a DMP candidate. Should match <code>base_config.min_coverage</code> .
<code>min_samples_abs</code>	<code>int</code>	1	1	Absolute minimum number of samples per locus per centroid.
<code>min_samples_pct</code>	<code>float</code>	0.05	[0, 1]	Minimum fraction of cohort samples that must cover a locus. Effective $\min = \max(\min_samples_abs, \text{ceil}(\min_samples_pct \times \text{cohort_size}))$.

Parameter	Type	Default	Constraints	Description
<code>delta_mean_reduction</code>	<code>float null</code>	<code>null</code>	<code>[0, 1]</code> or null	Optional pre-statistical gate: discard loci where $ \Delta\mu < \delta_{\min}$ before running the statistical test. Reduces test count for large CHH context runs.
<code>eps</code>	<code>float</code>	<code>1e-6</code>	<code>> 0</code>	Numerical floor for variance estimates to prevent division by zero.
<code>random_state</code>	<code>int</code>	<code>42</code>	<code>0</code>	RNG seed for reproducibility.

13.2.3 Biological Prioritization

Parameter	Type	Default	Constraints	Description
<code>effect_size_coverage</code>	<code>float</code>	<code>0.95</code>	<code>[0, 1]</code>	Per-context effect-mass filter: keep the minimum set of statistical DMPs covering this fraction of total effect mass (Eq. 4.2). <code>1.0</code> = keep all.
<code>biological_only_effect_size_coverage</code>	<code>float null</code>	<code>null</code>	<code>[0, 1]</code> or null	Optional rescue track: apply the same cumulative coverage to non-significant loci ($q > \alpha$) and flag them as biological-only DMPs.

Parameter	Type	Default	Constraints	Description
biological_only_max_candidates	int	5000	1	Maximum non-significant candidates per context to consider for biological-only rescue before ECDF rescoring.
max_tau2_for_dmp	float null	null	0 or null	Heterogeneity filter: drop loci where both groups exceed this between-sample variance estimate. Useful for removing subpopulation-driven loci.
lambda_var	float	2.0	[0, 20]	Variance penalty in the effect size formula: $e_i = \Delta\mu_i (1 - O_i) \exp(-\lambda_{\text{var}}(\sqrt{v_1} + \sqrt{v_2}))$
effect_size_use_mean_level	bool	true	—	Multiply effect size by a mean-level factor that down-weights loci at low average methylation (e.g. CHH context).
effect_size_mean_level_use_max	bool	true	—	Use $\max(\mu_1, \mu_2)$ (rather than the mean) for the mean-level factor.
effect_size_mean_level_weight	str	"saturating"	"sqrt", "linear", "saturating", "boundary"	Shape of the mean-level weight function.

Parameter	Type	Default	Constraints	Description
<code>effect_size_mean_level_k</code>	float	0.05	[0.01, 0.5]	Parameter k (saturating) or τ (boundary) for the mean-level weight.
<code>effect_size_mean_level_k_by_context</code>	dict or null	null	—	Per-context override of k (e.g. {"CHH": 0.20, "CHG": 0.10}). Useful for contexts with inflated raw effect sizes.

13.2.4 DMP Selection and Export

Parameter	Type	Default	Constraints	Description
<code>dmp_export_mode</code>	str	"dual"	"unified" or "dual"	dual: exports separate <code>dmps-{chrom}-discovery.csv</code> (for mapper/enricher) and <code>dmps-{chrom}-classifier.csv</code> (for classifier). unified: single CSV.
<code>dynamic_dmp_cutoff_enabled</code>	bool	true	—	Apply elbow detection on the effect-size distribution to trim the long tail of low-importance DMPs.
<code>dynamic_dmp_cutoff_relaxation</code>	float	1.0	> 0	Multiplier on the elbow threshold. < 1 keeps more DMPs; > 1 is stricter.

Parameter	Type	Default	Constraints	Description
discovery_dynamic_dmp_cutoff_enabled	boolean	false	—	Apply the elbow trim to the discovery export as well (default: discovery keeps all biologically filtered DMPs).
classifier_dmp_selection	string	"elbow"	"elbow" or "featurecuts_validation"	How to select the DMPs on panel. elbow: effect-size distribution trim. featurecuts_validation: optimize balanced accuracy on validation samples.
featurecuts_exhaustive_search	boolean	false	—	More thorough search over candidate panel sizes in featurecuts_validation mode (slower).
featurecuts_max_candidates	integer	50	1	Maximum number of panel size candidates evaluated in featurecuts mode.
featurecuts_max_k	integer null	null	1 or null	Upper bound on panel size k in featurecuts mode.
min_selected_dmps	integer null	null	1 or null	Minimum panel size (binary search floor).
min_dmps_for_export	integer	1000	1	Minimum rows in the exported DMP CSV. In unified mode, the CSV is widened to this count if the classifier elbow subset is smaller and more biological DMPs exist.

Parameter	Type	Default	Constraints	Description
<code>effect_size_weight_top</code>	float	1.0	[0.1, 5]	Power applied to normalized effect sizes when computing classifier feature weights: $w_i = (e_i/e_{\max})^p$. Values > 1 concentrate weight on top positions.
<code>filter_funnel_explore</code>	dict or null	null	—	If set, sweeps <code>effect_size_coverage</code> over a range and writes <code>filter_funnel.csv</code> . Dict with sub-key <code>effect_size_coverage</code> : {min, max, step}.

13.2.5 Context Weighting

Parameter	Type	Default	Constraints	Description
<code>use_context_weighting</code>	bool	true	—	Enable trimmed-mean context weighting when multiple contexts (CG, CHG, CHH) are processed together.
<code>trimmed_percentile_low</code>	float	0.10	[0, 0.5]; sum with <code>high</code> < 1	Remove the bottom this fraction of DMPs (by effect size) when computing the trimmed mean for context weighting.
<code>trimmed_percentile_high</code>	float	0.01	[0, 0.5]	Remove the top this fraction.

Parameter	Type	Default	Constraints	Description
<code>context_weight_direction</code>	string	"inverse"	"inverse" or "proportional"	inverse: weight 1/trimmed_mean (down-weight contexts with inflated effects, e.g. CHH). proportional: weight trimmed_mean.

13.2.6 Classifier Handoff and Sample Coverage

Parameter	Type	Default	Constraints	Description
<code>min_sample_coverage</code>	int	10	[1, 100]	Minimum coverage required at a DMP locus in a new sample being classified. Positions below this threshold are treated as missing. Should match base_config.min_coverage (see § 4.10).
<code>validation_min_coverage</code>	int	4	[1, 100]	Minimum coverage when extracting methylation from validation or centroid samples during the internal self-check.
<code>min_validation_coverage_per_position</code>	int	1	[1, 100]	Minimum number of validation samples that must cover a position for it to be used in calibration.

Parameter	Type	Default	Constraints	Description
<code>classifier_coverage_weighting</code>	bool	<code>true</code>	—	Weight the log-likelihood ratio by sample precision (higher coverage → more weight).
<code>temperature</code>	float	<code>1.0</code>	<code>[0.1, 10]</code>	Softmax temperature for the binary ECDF classifier metadata.
<code>enable_platt_calibration</code>	bool	<code>false</code>	—	Enable Platt scaling calibration on validation data.
<code>centroid_self_check_top_k</code>	int	<code>null</code>	1 or null	If set, restrict the self-check centroid classification to the top-K DMPs by effect size.

13.2.7 Debug and Output

Parameter	Type	Default	Constraints	Description
<code>debug</code>	bool	<code>false</code>	—	Enable verbose debug logging.
<code>use_gpu</code>	bool	<code>true</code>	—	Use GPU acceleration.
<code>export_sample_size_estimate</code>	bool	<code>false</code>	—	Add <code>n_estimated_per_group</code> column to the DMP CSV: estimated per-group sample size for target statistical power.
<code>target_power</code>	float	<code>0.8</code>	<code>[0.5, 0.999]</code>	Target power for sample size estimation (only used when <code>export_sample_size_estimate</code> is <code>true</code>).

Parameter	Type	Default	Constraints	Description
<code>synthetic_config</code>	dict	(see defaults)	<code>realism_level</code> "off", "basic", "advanced"	Configuration for synthetic ECDF validation sample generation. Sub-keys: <code>realism_level</code> , <code>avg_coverage</code> , <code>min_coverage</code> , <code>correlation_strength</code> , <code>variability_scale</code> , <code>add_missing</code> .

13.3 `step_config.detection` — Multiclass Export-Only Parameters

These parameters are **stripped before** `MethylDetectorConfig` is constructed; they are passed to the native multiclass PKL export step (`build_multiclass_model`). They have no effect on DMP detection.

Parameter	Type	Default	Constraints	Description
<code>multiclass_train_learned_head</code>	bool	false	—	If true , fit a multinomial logistic regression on top of the histogram-based pre-softmax scores during multiclass PKL export.
<code>multiclass_learned_logistic_C</code>	float	1.0	> 0	Inverse regularization strength for the logistic regression ($C = 1/\lambda$). Smaller values = stronger regularization.
<code>multiclass_learned_max_iter</code>	int	2000	1	Maximum iterations for the LBFGS solver.

Parameter	Type	Default	Constraints	Description
<code>multiclass_learned_logits_standardize</code>	bool	True	—	StandardScaler on the pre-softmax feature scores before logistic regression.
<code>multiclass_learned_logits_random_state</code>	int	0	0	RNG seed for the logistic regression solver.
<code>multiclass_learned_logits_class_weight</code>	dict or str	null	"balanced" or null	Class weights for the logistic regression. "balanced" computes $w_k = n_{\text{total}} / (K n_k)$. Recommended for imbalanced cohorts. See § 5.9.

13.4 step_config.mapper

See Chapter 7 for theoretical background.

Parameter	Type	Default	Description
<code>csv_pattern</code>	str	"dmps-*-* discovery.csv"	Glob pattern for DMP CSV files to map. Use <code>dmps-*-* classifier.csv</code> to map the classifier panel instead.
<code>gtf</code>	str (path)	required	Path to the GTF annotation file (e.g. GENCODE v44).
<code>upstream_size</code>	int	5000	Promoter window upstream of TSS (bp).
<code>downstream_size</code>	int	2000	Terminator window downstream of gene end (bp).

Parameter	Type	Default	Description
<code>min_intron_size</code>	int	0	Minimum intron size to include in mapping (bp).
<code>use_sp_regions</code>	bool	<code>true</code>	Use special-purpose genomic regions (promoter, terminator, etc.) in addition to exon/intron/gene body.
<code>storey_lambda</code>	float	0.4	Lambda parameter for Storey q-value estimation of gene-level p-values.
<code>w_promoter</code>	float	2.0	Weight for DMPs in promoter regions when computing gene-level DMP scores.
<code>w_terminator</code>	float	0.5	Weight for terminator region DMPs.
<code>w_gene_body</code>	float	1.0	Weight for gene body DMPs.
<code>w_exon</code>	float	1.5	Weight for exon DMPs.
<code>w_intron</code>	float	0.7	Weight for intron DMPs.
<code>w_unknown</code>	float	1.0	Weight for DMPs in unannotated or intergenic regions.
<code>disease_term</code>	str	<code>" "</code>	Disease term for literature/database enrichment (e.g. <code>"Prostate Cancer"</code>).
<code>enrich_disease</code>	bool	<code>false</code>	Run disease association enrichment alongside genomic mapping.
<code>enrich_source</code>	str	<code>"opentargets"</code>	Enrichment source: <code>"opentargets"</code> , <code>"grok"</code> , or <code>"grok+opentargets"</code> .
<code>enrich_profile</code>	str	<code>"permissive"</code>	Enrichment stringency profile.
<code>grok_max_workers</code>	int	1	Parallel workers for Grok API calls.

Parameter	Type	Default	Description
<code>grok_batch_size</code>	int	16	Gene batch size per Grok API request.
<code>rate_limit_delay</code>	float	5.0	Seconds to wait between Grok API calls.
<code>grok_max_retries</code>	int	3	Maximum Grok API retries per request.
<code>grok_429_cooldown</code>	int	60	Seconds to wait after a 429 rate-limit response.
<code>grok_batch_api</code>	bool	false	Use the Grok batch API instead of synchronous calls.
<code>grok_batch_poll_interval</code>	int	30	Polling interval (seconds) for batch API job completion.
<code>grok_batch_submit_chunk_size</code>	int	100	Genes per batch API submit chunk.
<code>grok_cache_ttl_days</code>	int	7	TTL (days) for cached Grok responses.
<code>optimize_dmps</code>	bool	false	Run DMP optimization before mapping.
<code>extend_after_stable</code>	bool	false	Extend the DMP list after stability analysis.
<code>run_bedtools_closest</code>	bool	false	Use bedtools <code>closest</code> for nearest-gene assignment.
<code>closest_gene_bed</code>	str null	null	BED file for bedtools <code>closest</code> gene lookup.

13.5 `step_config.enricher`

See [Chapter 8](#) for theoretical background.

Parameter	Type	Default	Description
<code>gene_column</code>	str	"gene_name"	Column in mapper output used as gene identifier.

Parameter	Type	Default	Description
disease_only	bool	true	Restrict enrichment to disease-associated genes from DisGeNET/Open Targets.
disease_association_type	list[str]	["direct", "indirect"]	Association types to include.
min_disease_evidence_level	str	"medium"	Minimum evidence level: "low", "medium", "high".
min_disease_score	float	0.0	Minimum disease association score.
min_dmp_count	int	2	Minimum number of mapped DMPs per gene.
min_unique_dmps	int null	null	Minimum unique (chromosome, position) DMP count per gene.
max_gene_q_value	float	0.05	Maximum gene-level q-value to include in enrichment.
sort_by	str	"total_weight"	Column to sort the enriched gene list by.
sort_ascending	bool	false	Sort direction.
library_preset	str null	null	Named Enrichr preset. Supported values include "cancer-core" and "cancer-extended". Used when libraries is not set.
libraries	list[str]	["KEGG_2021_Human", "Reactome_2022", "GO_Biological_Process_2023", "GO_Molecular_Function_2023", "GO_Cellular_Component_2023", "MSigDB_Hallmark_2020", "WikiPathway_2023_Human"]	Explicit Enrichr libraries for presentation. Precedence: if set, overrides library_preset.
top	int	150	Maximum number of genes to report.
cutoff	float	0.05	Adjusted p-value cutoff for pathway enrichment.
organism	str	"Human"	Organism for Enrichr queries.

Parameter	Type	Default	Description
<code>modules</code>	<code>bool</code>	<code>true</code>	Compute gene-gene co-citation modules.
<code>similarity_threshold</code>	<code>float</code>	<code>0.15</code>	Jaccard similarity threshold for module graph edges.
<code>cluster_resolution</code>	<code>float</code>	<code>0.8</code>	Leiden clustering resolution for module detection.
<code>module_cluster_max_q</code>	<code>float null</code>	<code>null</code>	Optional pre-clustering filter: keep only terms with Adjusted P-value $\leq$ <code>module_cluster_max_q</code> when building pathway modules. Recommended with large presets like <code>cancer-extended</code> .
<code>module_cluster_top_terms_per_library</code>	<code>int null</code>	<code>null</code>	Optional pre-clustering cap: keep top N enrichment terms per library (after q-value sorting) before module clustering.
<code>network_plot</code>	<code>str</code>	<code>"plotly"</code>	Library used for the network visualization (" <code>plotly</code> " or " <code>matplotlib</code> ").

13.6 `step_config.classifier`

See [Chapter 5](#) for theoretical background.

Parameter	Type	Default	Description
<code>ovr_binary_pickles_from_comparisons</code>		<code>false</code>	Build the OvR multiclass PKL automatically from per-comparison detection directories. Recommended for hierarchical multiclass projects.
<code>weight_method</code>	<code>str</code>	<code>"effect_size"</code>	Chromosome weight method: <code>"config"</code> , <code>"effect_size"</code> , <code>"linear_fitted"</code> , <code>"logistic_fitted"</code> , <code>"elasticnet_fitted"</code> .
<code>weight_fit_regularization</code>		<code>"lasso"</code>	Regularization for fitted weights: <code>"lasso"</code> , <code>"ridge"</code> , <code>"elasticnet"</code> .
<code>weight_fit_alpha</code>	<code>float</code>	<code>1.0</code>	Regularization strength for fitted weights.
<code>weight_fit_l1_ratio</code>	<code>float</code>	<code>0.5</code>	L1 ratio for elastic-net weight fitting.
<code>temperature</code>	<code>float</code>	<code>1.0</code>	Softmax temperature applied to classifier outputs.
<code>enable_platt_calibration</code>	<code>bool</code>	<code>false</code>	Apply Platt scaling calibration to binary classifier outputs.
<code>use_isotonic_calibration</code>	<code>bool</code>	<code>false</code>	Enable isotonic probability calibration. During labeled runs, calibrators are fit and then applied; during inference, saved calibrators are reused when present.
<code>use_elasticnet_stacking</code>	<code>bool</code>	<code>false</code>	Fit chromosome weights with ElasticNet stacking on labeled data.

Parameter	Type	Default	Description
calibration_train_fraction	float null	null	If set in (0,1], fit isotonic/stacking on a stratified per-class fraction (MC-style holdout mask). null means use full labeled set.
calibration_seed	int null	null	Seed for stratified calibration/stacking fit mask.
trimmed_percentile_low	float	0.10	Lower percentile for trimmed-mean effect-size weights.
trimmed_percentile_high	float	0.01	Upper percentile for trimmed-mean effect-size weights.
chromosome_weights	dict null	null	Fixed per-chromosome weights dict (bypasses trimmed-mean computation).
debug	bool	false	Verbose debug logging.
log_level	str	"INFO"	Log level: "DEBUG", "INFO", "WARNING", "ERROR".
panel	dict null	null	Hierarchical prediction panel for OvR readout. Sub-keys: primary_family (str), families (dict: family → list of class names), indeterminate_delta (float).

13.7 step_config.predictor

Parameter	Type	Default	Description
debug	bool	false	Verbose debug logging for the predictor.

Parameter	Type	Default	Description
<code>panel</code>	<code>dict</code> <code>null</code>	<code>null</code>	Same hierarchical panel as classifier. Usually identical to <code>step_config.classifier.panel</code> .
<code>multiclass_model_path</code>	<code>str</code> <code>null</code>	<code>null</code>	Override path to a specific multiclass PKL. If <code>null</code> , resolved from the project classifier output.

Note. Do not repeat the `controls/diseases` block inside `step_config.predictor`. Sample paths are inherited from the project’s top-level cohort structure. See Chapter 11 for the correct anti-pattern avoidance.

13.8 `step_config.validation`

Fields inherited from the project (`samples_base_path`, `output_base`, cohort structure) do not need to be repeated here.

Parameter	Type	Default	Constraints	Description
<code>train_fraction</code>	<code>float</code>	<code>required</code>	<code>(0, 1)</code> exclusive	Fraction of each cohort used for training per iteration.
<code>n_iterations</code>	<code>int</code>	<code>required</code>	<code>1</code>	Number of Monte Carlo iterations.
<code>seed</code>	<code>int</code> <code>null</code>	<code>null</code>	—	Base RNG seed; iteration r uses <code>seed + r</code> .
<code>run_stability</code>	<code>bool</code>	<code>false</code>	—	Run stability analysis after the main loop.
<code>stability_dmp_freq</code>	<code>float</code>	<code>0.7</code>	<code>[0, 1]</code>	Minimum recurrence frequency τ for a DMP to be declared stable.

Parameter	Type	Default	Constraints	Description
<code>stability_min_balanced_accuracy</code>	<code>float</code>	<code>null</code>	<code>[0, 1]</code> or <code>null</code>	Only runs with <code>balanced_accuracy</code> this value contribute to DMP frequency counts.
<code>stability_gene_frequency</code>	<code>float</code>	<code>0.5</code>	<code>[0, 1]</code>	Minimum recurrence frequency for a gene (enricher output) to be declared stable.
<code>run_mapper_and_enricher</code>	<code>bool</code>	<code>false</code>	—	Legacy field; MC iterations now run <code>centroid+detector</code> only. Mapper/enricher are part of <code>--freeze</code> .
<code>skip_enricher</code>	<code>bool</code>	<code>false</code>	—	Skip the enricher step even when <code>run_mapper_and_enricher</code> is <code>true</code> .
<code>freeze_stable_dmp_csv</code>	<code>str</code>	<code>null</code>	—	Path to the stable DMP CSV for <code>--freeze</code> . Defaults to <code>monte_carlo_runs/stability/</code>
<code>production_outputs_dir</code>	<code>str</code>	<code>null</code>	—	Override directory for the production freeze output. Defaults to <code>monte_carlo_runs/production/</code>
<code>predictor_only</code>	<code>bool</code>	<code>false</code>	—	If <code>true</code> , only run methyl-predictor per iteration (Workflow 2).
<code>frozen_project_path</code>	<code>str</code> <code>null</code>	<code>null</code>	—	Path to <code>production/project.json</code> for predictor-only mode. Defaults to <code>monte_carlo_runs/production/</code> if it exists.

Parameter	Type	Default	Constraints	Description
<code>require_biological_review_for_model</code>	<code>bool</code>	<code>false</code>	—	If <code>true</code> , <code>methylation_validation</code> <code>--model</code> is blocked unless <code>biological_review_confirmed</code> .
<code>biological_review_confirmed</code>	<code>bool</code>	<code>false</code>	—	Explicit confirmation flag that unlocks <code>--model</code> when biological review is required.
<code>abort_on_step_failure</code>	<code>bool</code>	<code>false</code>	—	If <code>true</code> , abort all iterations when any pipeline step fails. If <code>false</code> , log the error and skip the iteration.
<code>model_backend</code>	<code>str</code>	<code>"ecdf"</code>	<code>"ecdf", "tabular_sklearn", "generative_hybrid"</code>	Backend for <code>--model</code> . <code>ecdf</code> : Classifier → predictor. <code>tabular_sklearn</code> : bundle + sklearn train/predict. <code>generative_hybrid</code> : bundle + latent-density generative train/predict.
<code>model_bundle_dir</code>	<code>str null</code>	<code>null</code>	—	Optional output directory for model feature bundle artifacts. Default: <code><production_output_dir>/model</code>
<code>model_weight_column</code>	<code>str</code>	<code>"weight"</code>	—	Preferred detector DMP column used as feature weight in model bundle construction; falls back to <code>effect_size</code> or <code>1.0</code> .

Parameter	Type	Default	Constraints	Description
tabular_model_type	str	"random_forest"	"random_forest", "hist_gradient_boosting", "logistic_regression"	Model type when model_backend="tabular_sklearn"
tabular_max_dmps	int	5000	10	Maximum number of DMP loci retained from bundle index for tabular/genera- tive backend training.
covariates_path	str null	null	—	Optional covariates sidecar (.h5 preferred, .csv accepted) for tabular or generative backend.
covariate_id_column	str	"sample_id"	—	Covariates column used to join rows with sample basenames.
generative_latent_dim	int	16	2	Latent dimensionality for model_backend="generative_hybrid"
generative_kl_weight	float	0.1	0	KL-like regularization weight metadata/control for generative hybrid training.
generative_density_type	str	"diag_gaussian"	currently only "diag_gaussian"	Latent class-density family for the generative hybrid backend.
generative_epochs	int	50	1	Epoch count metadata/control for generative hybrid training.
generative_batch_size	int	64	1	Batch size metadata/control for generative hybrid training.

Parameter	Type	Default	Constraints	Description
<code>generative_seed</code>	<code>int</code>	<code>13</code>	—	RNG seed for generative hybrid training.
<code>generative_calibration</code>	<code>bool</code>	<code>false</code>	—	Enable optional calibration stage for generative hybrid probabilities when available.
<code>generative_covariates_strict</code>	<code>bool</code>	<code>true</code>	—	If <code>true</code> , require all sample IDs to exist in covariates sidecar during generative backend joins.

13.9 `step_config.progression`

See [Chapter 12](#) for workflow placement after freeze.

Parameter	Type	Default	Description
<code>enabled</code>	<code>bool</code>	<code>false</code>	Run <code>methyl-disease-progression</code> after enricher during <code>--freeze</code> .
<code>output_dir</code>	<code>str</code> <code>null</code>	<code>null</code>	Optional output directory override. Default: <code><project_root>/progression</code> .
<code>ordered_comparison_labels</code>	<code>list[str]</code> <code>null</code>	<code>null</code>	Explicit stage order using comparison labels and/or disease groups.
<code>ordered_disease_groups</code>	<code>list[str]</code> <code>null</code>	<code>null</code>	Alias for stage order override.
<code>strict_missing</code>	<code>bool</code>	<code>false</code>	If <code>true</code> , progression fails when expected stage files are missing.

Parameter	Type	Default	Description
<code>report_md</code>	<code>bool</code>	<code>false</code>	Emit <code>progression/report.md</code> in addition to CSV/JSON outputs.

13.10 `step_config.alignment_qc`

See [Chapter 9](#) for theoretical background.

Parameter	Type	Description
<code>groups</code>	<code>list[str] null</code>	Cohort group labels to include (e.g. <code>["control", "disease"]</code>). <code>null</code> = all groups.
<code>output_dir</code>	<code>str null</code>	Override output directory.
<code>validate_schema</code>	<code>bool</code>	Validate per-sample QC JSON against the canonical schema.

13.11 Configuration Audit Matrix

The active-steps configuration audit (declared vs consumed vs inherited keys, alias/legacy status, evidence paths, and redundancy dispositions) is maintained in:

- [docs/config_parameter_matrix.md](#)

Use that matrix as the practical source for cleanup/deprecation planning. This chapter remains the canonical reference for currently supported keys.

Part IV

Part IV: User Guide

14 User Guide

This chapter is a practical end-to-end guide. For theoretical background, see Part I. For the complete parameter reference, see [Chapter 13](#).

14.1 Prerequisites

14.1.1 Software requirements

- Python 3.10 or later
- All MethylPipeline packages installed in a virtual environment
- The following CLI tools must be on PATH inside the environment:
 - methyl-centroid
 - methyl-detector
 - methyl-classifier
 - methyl-predictor
 - methyl-mapper
 - methyl-enricher
 - methyl-disease-progression
 - methyl-validation

14.1.2 Environment setup

From the MethylPipeline repository root:

```
# Create and activate the virtual environment
python3.12 -m venv .venv
source .venv/bin/activate

# Install all packages in editable mode (development)
pip install -e packages/methylutils
pip install -e packages/methylcentroid
pip install -e packages/methyldetector
pip install -e packages/methylclassifier
pip install -e packages/methylpredictor
pip install -e packages/methylmapper
pip install -e packages/methylenricher
```

```
pip install -e packages/methyldiseaseprogression
pip install -e packages/methylvalidation
```

After installation, verify the CLI tools are available:

```
methy-validation --help
methy-detector --help
```

14.1.3 Activating the environment in subsequent sessions

```
cd /path/to/MethylPipeline
source .venv/bin/activate
```

The virtual environment must be activated before running any `methy-*` command. The repository rule is documented in `.cursor/rules/python-venv.mdc`.

14.2 Creating a Project Configuration File

The project configuration file is the single entry point for all pipeline steps. See Chapter 11 for the full anatomy. A minimal template for a binary (control vs disease) project:

```
{
  "project_name": "my_project",
  "laboratory": "my_lab",
  "batch": "batch_1",
  "output_base": "/path/to/output",
  "samples_base_path": "/path/to/samples",
  "chromosomes": ["1", "2", "3", "4", "5", "6", "7", "8", "9", "10",
                  "11", "12", "13", "14", "15", "16", "17", "18", "19",
                  "20", "21", "22", "X"],
  "contexts": ["CG"],
  "comparisons": "control_vs_each_disease",
  "controls": {
    "label": "healthy",
    "groups": [
      { "label": "all", "sample_paths": ["configs/healthy.csv"] }
    ]
  },
  "diseases": {
    "label": "disease",
    "groups": [
```

```

    { "label": "disease", "sample_paths": ["configs/disease.csv"] }
  ]
},
"step_config": {
  "centroid": {
    "base_config": {
      "min_coverage": 4,
      "binned_stats_bins": 20,
      "use_gpu": true
    }
  },
  "detection": {
    "alpha": 0.05,
    "min_coverage": 4,
    "min_sample_coverage": 4,
    "effect_size_coverage": 0.95
  },
  "classifier": {
    "ovr_binary_pickles_from_comparisons": true
  },
  "predictor": {
    "debug": false
  },
  "validation": {
    "train_fraction": 0.8,
    "n_iterations": 20,
    "seed": 42,
    "run_stability": true,
    "stability_dmp_freq": 0.7
  }
}
}

```

For multiclass hierarchical projects (e.g. healthy vs multiple disease stages), see the annotated example in [Chapter 11](#).

14.2.1 Cohort CSV format

Each cohort CSV (`healthy.csv`, `disease.csv`, etc.) lists sample identifiers, one per line, with an optional `sample` header:

```

sample
sample_001
sample_002
sample_003

```

Sample paths are resolved as `<samples_base_path>/<sample_name>`.

14.3 Workflow 1: Model Creation

Full theoretical background: § [12.2](#).

14.3.1 Stage-oriented execution checklist

Workflow 1 should be run and reviewed in three explicit stages:

1. **Stability** (`--stability`): optimize panel robustness (`n_iterations`, `train_fraction`, `stability_dmp_freq`) and verify `stable_dmps_production.csv` is non-empty and biologically plausible.
2. **Freeze/analysis** (`--freeze`): lock the panel into `production/project.json`, then review production detector outputs together with mapper/enricher summaries.
3. **Modeling** (`--model`): train final classifier and predictor artifacts only after stability and biological review are complete.

Treat changes in any of these stage inputs as a model version change, not as a minor rerun.

14.3.2 Step 1: Run Monte Carlo with stability

```
source .venv/bin/activate
methyl-validation --project configs/my_project.json --stability
```

This runs `n_iterations` Monte Carlo discovery cycles (centroid → detector), then aggregates DMP recurrence frequencies.

Monitoring progress. Progress is shown via a Rich progress bar in interactive terminals. Iteration logs are written to `monte_carlo_runs/run_000N/logs/`.

Useful flags: - `--iterations N` — override `n_iterations` from config for a quick test run - `--seed S` — override the seed - `--skip-enricher` — skip enricher during `--freeze` (and progression, if enabled)

14.3.3 Step 2: Check stability results

After the loop completes, review the stability output:

```
cat monte_carlo_runs/stability/stability_summary.json
head monte_carlo_runs/stability/stable_dmps_production.csv
```

The summary reports: - `n_runs_analyzed` — number of qualifying runs - `total_unique_dmps` — total unique positions across all runs - `stable_dmps_at_threshold` — positions with frequency `stability_dmp_freq`

If `stable_dmps_at_threshold` is very low (< 1000), consider reducing `stability_dmp_freq` or increasing `n_iterations`.

14.3.4 Step 3: Run the production freeze

```
methyl-validation --project configs/my_project.json --freeze
```

This runs the production freeze pipeline on all samples using the stable DMP panel (centroid → detector(fixed panel) → mapper → enricher). Output is in `monte_carlo_runs/production/`.

If `step_config.progression.enabled=true`, freeze also runs methyl-disease-progression and writes `production/progression/`.

14.3.5 Step 4: Run the final model stage

```
methyl-validation --project configs/my_project.json --model
```

This runs classifier → predictor on the frozen production project.

If `step_config.validation.require_biological_review_for_model=true`, `--model` requires:

```
"biological_review_confirmed": true
```

14.3.6 Choosing the `--model` backend

By default, `--model` uses the ECDF backend (`model_backend: "ecdf"`). Advanced projects can choose a different backend in `step_config.validation`:

- `model_backend: "tabular_sklearn"`
- `model_backend: "generative_hybrid"`

CLI overrides are also available in `methy1-validation` for quick experiments (for example `--model-backend generative_hybrid`, `--generative-latent-dim`, `--generative-density-type`, `--generative-calibrate`).

Example:

```
"validation": {
  "model_backend": "generative_hybrid",
  "model_weight_column": "weight",
  "tabular_max_dmps": 5000,
  "generative_latent_dim": 16,
  "generative_density_type": "diag_gaussian",
  "generative_epochs": 50,
  "generative_batch_size": 64,
  "generative_seed": 13
}
```

14.3.7 Backend comparison (practical)

Backend	Best used when	Main strengths	Main risks / caveats
<code>ecdf</code>	You want continuity with historical MethylPipeline behavior and detector-classifier semantics	Native pipeline path, direct compatibility with existing classifier/predictor tooling, easiest interpretation relative to prior studies	Can underperform when class boundaries need stronger discriminative reshaping or additional covariates
<code>tabular_sklearn</code>	You want fast supervised baselines or covariate-aware tabular modeling from fixed DMP bundles	Simple deployment, mature sklearn ecosystem, often strong baseline performance with moderate tuning	Less directly tied to ECDF likelihood semantics; performance may depend strongly on feature cap (<code>tabular_max_dmps</code>) and model choice
<code>generative_hybrid</code>	You want likelihood-based latent modeling with posterior probabilities while preserving detector-driven weighting	Hybrid latent-density scoring, consistent DMP bundle contract, supports binary and multiclass posterior inference	More modeling complexity; requires careful reporting of latent/density settings and calibration choices for reproducibility

For manuscript methods sections, always report backend choice, key hyperparameters, and whether covariates were used.

14.3.8 Optional: Run progression synthesis as a standalone step

If your project has multiple ordered disease groups and you want a dedicated cross-stage summary after freeze, run:

```
methy1-disease-progression --project configs/my_project.json
```

Useful flags:

- `--ordered-comparison-labels` `pca_pca1,pca_pca2,pca_pca3,pca_pca4` to enforce explicit stage order
- `--strict-missing` to fail when any stage input file is missing
- `--report-md` to also emit a markdown summary report

Default output directory is `production/progression/` when called from the freeze workflow, or `<project_root>/progression` for standalone runs.

14.4 Workflow 2: Model Use for Prediction

Full theoretical background: § [12.3](#).

14.4.1 Prerequisites

`monte_carlo_runs/production/project.json` must exist (created by Workflow 1's `--freeze` step).

14.4.2 Run predictor-only evaluation

```
methy1-validation --project configs/my_project.json --predictor-only
```

This applies the frozen production model to random held-out splits. The same `n_iterations` from `step_config.validation` is used.

Performance comparison. Workflow 2 is much faster than Workflow 1 because each iteration runs only `methy1-predictor` rather than the full pipeline. On typical hardware, a 20-iteration Workflow 2 run takes minutes rather than hours.

14.5 Running Individual Pipeline Steps

Each pipeline step can be run independently using its `--project` flag:

```
# Rebuild centroids for all groups
methyl-centroid --project configs/my_project.json --group all

# Re-run detection
methyl-detector --project configs/my_project.json

# Re-run classification
methyl-classifier --project configs/my_project.json

# Re-run prediction (requires val CSVs)
methyl-predictor --project configs/my_project.json

# Re-run mapping
methyl-mapper --project configs/my_project.json

# Re-run enrichment
methyl-enricher --project configs/my_project.json
```

For multi-comparison (hierarchical) projects, use `--per-cancer-group` with `methyl-detector` and `methyl-classifier`:

```
methyl-detector --project configs/my_project.json --per-cancer-group
methyl-classifier --project configs/my_project.json --per-cancer-group
```

14.6 Output Directory Layout

Given `output_base = "/work/prostate-cancer"` and `project_name = "Healthy_vs_PCa1-4-CG"`:

```
/work/prostate-cancer/Healthy_vs_PCa1-4-CG/
  centroids/
    controls/healthy/all/1-CG.h5 (and other chromosomes)
    diseases/cancer/pca_pca1/1-CG.h5 (and pca2, pca3, pca4)
  detections/
    all/pca_pca1/
      dmeps-1-discovery.csv
      dmeps-1-classifier.csv
      classifier-1-CG.pkl
  classifiers/
```

```

all/
  pca_pca1/run_0001-classifier.pkl
  multiclass-classifier.pkl
  classifier_all_run_0001.pkl
mapper/
  all/pca_pca1/all-gene_name-combined.csv
enricher/
  all/pca_pca1/enrichment_merged.csv
monte_carlo_runs/
  run_0001/ ... run_000N/
    project.json
    training_all.csv / testing_all.csv (or per-group CSVs in multiclass)
    centroids/
    detections/
    logs/
  all_metrics.csv           ← one row per iteration
  metrics_summary.json      ← mean, std, percentiles
  step_timings.csv          ← duration per step per run
  stability/
    stable_dmps_production.csv
    stability_summary.json
  production/
    project.json            ← frozen project with fixed_dmp_panel
    centroids/
    detections/
    mapper/
    enricher/
    progression/            ← optional when step_config.progression.enabled=true
    classifiers/
    predictors/
    production_summary.json

```

14.7 Interpreting Results

14.7.1 all_metrics.csv

One row per successful iteration. Key columns:

Column	Description
iteration	Iteration index (1-based)
run_id	Directory name (e.g. run_0001)
accuracy	Overall accuracy

Column	Description
<code>balanced_accuracy</code>	Primary metric: mean per-class recall. More informative than accuracy for imbalanced datasets.
<code>sensitivity</code>	Recall for the positive class (binary mode)
<code>specificity</code>	Recall for the negative class (binary mode)
<code>macro_f1</code>	Macro-averaged F1 across all classes
<code>n_samples</code>	Total validation samples
<code>n_classes</code>	Number of classes

14.7.2 `metrics_summary.json`

For each metric in `all_metrics.csv`, reports:

```
{
  "balanced_accuracy": {
    "mean": 0.75,
    "std": 0.05,
    "min": 0.62,
    "max": 0.84,
    "count": 20,
    "p5": 0.65,
    "p25": 0.72,
    "p50": 0.76,
    "p75": 0.79,
    "p95": 0.82
  }
}
```

Interpreting balanced accuracy. For K classes, random chance gives $1/K$ (0.2 for 5 classes). A production model with mean BA 0.75 and std 0.05 is substantially above chance. Publication claims should report mean $\pm$ std and the 95th percentile interval.

14.7.3 `validation_metrics.json` (per run)

Located at `run_000N/predictors/validation_metrics.json`. Contains:

- `accuracy`, `balanced_accuracy`, `confusion_matrix`, `per_class` metrics
- `sample_dmp_coverage.dmps_used_fraction_median` — **critical diagnostic field** (see below)
- `n_samples`, `n_classes`, `class_names`

14.7.4 Confusion matrix interpretation

The confusion matrix is indexed `[true_class][predicted_class]`. A model biased toward the majority class (e.g. healthy) shows high recall for class 0 (`all`) and near-zero recall for all other classes. This is the primary symptom of class imbalance (§ 12.5).

14.8 Three Key Diagnostics

14.8.1 Diagnostic 1: Low DMP Coverage

Symptom: `sample_dmp_coverage.dmps_used_fraction_median < 0.5` in `validation_metrics.json`. All cancer classes predicted as healthy.

Root cause: `min_sample_coverage` in `step_config.detection` is higher than the centroid `min_coverage` in `step_config.centroid.base_config`.

Fix: Align both thresholds:

```
"centroid": { "base_config": { "min_coverage": 4 } },
"detection": { "min_coverage": 4, "min_sample_coverage": 4 }
```

Verification: After re-running, check `dmps_used_fraction_median` in the new run's `validation_metrics.json`. Expected range: 0.6–0.95.

See § 4.10 for theoretical background.

14.8.2 Diagnostic 2: Class Imbalance

Symptom: Confusion matrix shows recall = 1.0 for the majority class (e.g. healthy/control) and recall = 0 for all disease classes. Balanced accuracy close to 1/K.

Root cause: The training cohort has substantially more samples in one class. The multinomial logistic head defaults to the majority class when histogram scores are ambiguous.

Fix:

```
"detection": {
  "multiclass_train_learned_head": true,
  "multiclass_learned_class_weight": "balanced"
}
```

This sets $w_k = n_{\text{total}} / (Kn_k)$ in the logistic regression. Note that `multiclass_train_learned_head: true` is required for `multiclass_learned_class_weight` to take effect.

Verification: Re-run. The confusion matrix diagonal should become more balanced. Per-class recall should be non-zero for all classes.

See § 5.9 for theoretical background.

14.8.3 Diagnostic 3: Sklearn Version Mismatch

Symptom: `InconsistentVersionWarning` during prediction. The model was trained with sklearn X.Y.Z but is being run with A.B.C.

Check:

```
import pickle, sklearn
with open("monte_carlo_runs/production/classifiers/multiclass-classifier.pkl", "rb") as f:
    model = pickle.load(f)
print("Trained with sklearn:", model["metadata"].get("sklearn_version"))
print("Current sklearn:      ", sklearn.__version__)
```

Fix: Re-run `--freeze` with the current sklearn version. The new PKL will be trained with the current version and record it in metadata.

Long-term: Pin `scikit-learn` to a specific version in your environment to avoid future mismatches.

14.9 Running with Docker

If you use the MethylPipeline Docker container, paths in the project config must be valid **inside** the container. Mount your data volumes at the expected paths.

```
cd /path/to/MethylPipeline/docker
docker compose up -d

# Run Workflow 1 inside the container
docker exec -w /workspace methylpipeline \
    methyl-validation --project /workspace/configs/my_project.json --stability

# Run the production freeze
docker exec -w /workspace methylpipeline \
    methyl-validation --project /workspace/configs/my_project.json --freeze
```

Override `n_iterations` for a quick test:

```
docker exec -w /workspace methylpipeline \
  methyl-validation --project /workspace/configs/my_project.json --stability --iterations 3
```

14.10 Troubleshooting

Problem	Likely cause	Fix
methyl-validation: command not found	venv not activated	<code>source .venv/bin/activate</code>
Error: base_project not found	Wrong path in <code>step_config.validation.base_project</code> (legacy mode)	Use <code>--project</code> flag instead of <code>base_project</code>
Monte Carlo config needs at least two cohorts	Passing project file to <code>--config</code> instead of <code>--project</code>	Use <code>--project</code> <code>configs/my_project.json</code>
<code>step_config.validation</code> not found	Project file has no <code>step_config.validation</code> block	Add the validation block (see § 14.2)
<code>FileNotFoundError:</code> <code>fixed_dmp_panel</code> not found	<code>--freeze</code> was run before <code>--stability</code>	Run <code>--stability</code> first; stable DMP CSV must exist
Blind predictor: blind mode rejected	<code>step_config.predictor.blind</code> is set	Remove <code>blind</code> from predictor config for MC runs
Low iteration count in metrics	Some iterations failed due to missing CSVs or step errors	Check <code>run_*/logs/</code> for error messages; set <code>abort_on_step_failure:</code> <code>false</code> to skip bad runs
Stability panel is empty	All runs failed or DMP frequency threshold too high	Check <code>stability/stability_summary.json</code> ; lower <code>stability_dmp_freq</code>
<code>InconsistentVersionWarning</code> for LogisticRegression	sklearn version mismatch	Re-run <code>--freeze</code> ; see Diagnostic 3 above
Mapper fails with GTF error	GTF path wrong or file missing	Check <code>step_config.mapper.gtf</code> points to a valid GTF file

Blondel, Vincent D., Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. “Fast Unfolding of Communities in Large Networks.” *Journal of Statistical Mechanics: Theory and Experiment* 2008 (10): P10008. <https://doi.org/10.1088/1742-5468/2008/10/P10008>.

Chen, Edward Y., Christopher M. Tan, Yan Kou, et al. 2013. “Enrichr: Interactive and Collaborative HTML5 Gene List Enrichment Analysis Tool.” *BMC Bioinformatics* 14: 128. <https://doi.org/10.1186/1471-2105-14-128>.

- DerSimonian, Rebecca, and Nan Laird. 1986. “Meta-Analysis in Clinical Trials.” *Controlled Clinical Trials* 7 (3): 177–88.
- Domingos, Pedro, and Michael Pazzani. 1997. “On the Optimality of the Simple Bayesian Classifier Under Zero-One Loss.” *Machine Learning* 29 (2): 103–30.
- Feller, William. 1948. “On the Kolmogorov-Smirnov Limit Theorems for Empirical Distributions.” *The Annals of Mathematical Statistics* 19 (2): 177–89. <https://doi.org/10.1214/aoms/1177730243>.
- Fisher, Ronald A. 1925. *Statistical Methods for Research Workers*. Oliver; Boyd.
- Fritsch, F. N., and R. E. Carlson. 1980. “Monotone Piecewise Cubic Interpolation.” *SIAM Journal on Numerical Analysis* 17 (2): 238–46. <https://doi.org/10.1137/0717021>.
- Kolmogorov, A. N. 1933. “Sulla Determinazione Empirica Di Una Legge Di Distribuzione.” *Giornale Dell’Istituto Italiano Degli Attuari* 4: 83–91.
- Mann, H. B., and D. R. Whitney. 1947. “On a Test of Whether One of Two Random Variables Is Stochastically Larger Than the Other.” *The Annals of Mathematical Statistics* 18 (1): 50–60. <https://doi.org/10.1214/aoms/1177730491>.
- Ochoa, David, Andrew Hercules, Miguel Carmona, et al. 2023. “The Next-Generation Open Targets Platform: Reimagined, Redesigned, Rebuilt.” *Nucleic Acids Research* 51 (D1): D1353–59. <https://doi.org/10.1093/nar/gkac1046>.
- Piñero, Janet, Juan M. Ramírez-Anguita, Judit Saüch-Pitarch, et al. 2020. “The DisGeNET Knowledge Platform for Disease Genomics: 2019 Update.” *Nucleic Acids Research* 48 (D1): D845–55. <https://doi.org/10.1093/nar/gkz1021>.
- Platt, John C. 1999. “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods.” In *Advances in Large Margin Classifiers*. MIT Press.
- Storey, John D. 2003. “The Positive False Discovery Rate: A Bayesian Interpretation and the q-Value.” *The Annals of Statistics* 31 (6): 2013–35. <https://doi.org/10.1214/aos/1074290335>.
- Stouffer, Samuel A., Edward A. Suchman, Leland C. DeVinney, Shirley A. Star, and Robin M. Williams. 1949. *The American Soldier: Adjustment During Army Life*. Princeton University Press.